

LightMem-Ego Local Deployment — Technical Setup & Troubleshooting Report

Repository: <https://github.com/zjunlp/LightMem-Ego> **User:** tahagpt (Taha), FAST-NUCES Islamabad, FYP project **Machine:** Ubuntu laptop, `/home/tahagpt/Documents/fyp/LightMem-Ego` (also referenced earlier as `~/Documents/FYP/LightMem-Ego` — path casing changed partway through the conversation) **Backend working directory:** `~/Documents/fyp/LightMem-Ego/src/backend`

1. Project Overview

LightMem-Ego (backend package name: `em2mem-online-server`, internal package `em2mem`) is an online memory-management pipeline designed for egocentric (first-person, e.g. smart-glasses) video/audio streams. Based on what the repository's own files revealed during this session, the system:

- Ingests frame + audio streams from a session (camera frames, audio chunks), optionally from Rokid smart glasses via a phone SDK connector.
- Runs ASR (automatic speech recognition) via WhisperX on audio.
- Builds "current" memory (`M_cur`), "short-term" memory (`M_st`), an MST — micro-event short-term — structure, and long-term episodic/semantic memory.
- Uses a visual embedding model (VLM2Vec V2.0, built on a Qwen2-VL-2B backbone) to embed frames for visual search.
- Uses a text embedding model (Qwen3-Embedding-4B) for text-based retrieval (HippoRAG-based long-term retrieval).
- Uses an LLM (configurable, OpenAI-compatible) for: MST refinement, MST-to-episodic consolidation, memory building, evidence captioning, and final question answering.
- Exposes a FastAPI HTTP server (`api_server.py`) with endpoints for starting a stream session, uploading frames/audio, checking status, and asking questions (`/ask/{session_id}`), with both legacy-polling and SSE-streaming response modes.
- Runs as a main API process plus ~9 independent background workers (stream, live_ingest, preprocess, refine, consolidation, visual, memory, query, plus optional asr/asr_stream and a legacy evidence worker) that communicate via a task-queue-on-disk pattern (`online_tasks/`).

Why local: The user did not have an OpenAI API key and does not have a GPU, so the goal became running the entire pipeline — ASR, visual embeddings, text embeddings, and the LLM — on CPU-only hardware, using free/local substitutes wherever the reference implementation assumed paid APIs or GPU-hosted models.

Models ultimately involved (final working state):

- **Text LLM:** `llama3.2:3b` via Ollama (local)
- **Vision LLM:** `llava:7b` via Ollama (local) — after `llama3.2-vision` failed due to an Ollama bug
- **Visual embeddings:** `openai/clip-vit-base-patch32` (CLIP) — substitute for VLM2Vec V2.0/Qwen2-VL-2B
- **Text embeddings:** `sentence-transformers/all-MiniLM-L6-v2` — substitute for Qwen3-Embedding-4B
- **ASR:** WhisperX with `Systran/faster-whisper-small` — substitute for the originally-suggested `large-v3`, chosen for CPU speed

Models considered but **not** used in the final state: OpenAI GPT models (no API key), Mistral API, Groq API (`llama-3.3-70b-versatile`), `meta-llama/llama-4-scout-17b-16e-instruct`) — Groq was set up and partially working but the user later clarified the intent was to run everything locally, so the final state moved to Ollama.

2. Final Environment

Category	Details
Operating System	Ubuntu (Linux), user shell prompts show (tahagpt@TahaGPT) / (tahagpt@pcn)
OS Version	Unknown / not specified in chat
Hardware	No GPU ((nvidia-smi) not used to confirm, but user explicitly stated "i don't have the gpu")
CPU	Unspecified model; (free -h) showed 15Gi total RAM, system frequently near memory capacity during testing
GPU	None — confirmed explicitly by user
RAM	15832.2 MiB total (per (top)/(free -h) output near end of session); as low as 179.6 MiB free at one point; swap 3.7Gi total, up to 3.5Gi used
Python Version	3.10 (venvs built as (python3.10), e.g. (.venv/lib/python3.10/site-packages))
CUDA Version	N/A (CPU-only; various (CUDA_VISIBLE_DEVICES) env vars set by scripts but unused since (--device cpu) forced)
LLM Runtime	Ollama (systemd service), version (0.33.2) (confirmed via (ollama --version)); later required update via `curl -fsSL https://ollama.com/install.sh`
Models	((llama3.2:3b), (llama3.2-vision) (broken, abandoned), (llava:7b) (working)
Package Manager	(pip) (with (hf)/(huggingface-cli) for model downloads)
Project Directory	(~/Documents/fyp/LightMem-Ego/src/backend) (also seen as (~/Documents/FYP/LightMem-Ego/src/backend) earlier in the conversation — case changed)
API Server Port	(8000) ((EM2MEM_API_PORT=8000))
CLIP Embedding Server Port	(18091)
Text Embedding Server Port	(18096)
Ollama Port	(11434)
Database	None observed — file/JSON-based task queues under (online_tasks/), session data under (online_sessions/)
Other Dependencies	((ffmpeg), (ffprobe), (whisperx), (sentence-transformers), (fastapi), (uvicorn), (transformers==5.10.2) (pinned), (torch==2.12.0) (pinned), (torchvision==0.27.0) (pinned), (numpy==2.0.2) (pinned), (pandas==2.3.3) (pinned))

3. Initial Setup

3.1 Cloning the repository

```
bash

git clone https://github.com/zjunlp/LightMem-Ego.git
cd LightMem-Ego
```

Result: Not explicitly confirmed executed by the user in the transcript, but implied by all subsequent commands referencing the cloned directory.

3.2 Backend Python environment

Recommended initial setup:

```
bash
```

```
cd src/backend
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
```

3.3 Environment file

```
bash

cp .env.example .env
```

The `.env` file went through **many** rounds of editing over the course of the conversation (see Section 10 for the final reference).

3.4 Frontend (mentioned, not exercised in this conversation)

```
bash

cd src/frontend/online_web
npm install
npm run dev
```

Not actually run/tested in this conversation — the entire session focused on the backend and API-level testing via `curl`.

4. Installation History

Component: Hugging Face Hub CLI

- **Purpose:** Download models from Hugging Face
- **Installation method:** `pip`
- **Command:** `pip install huggingface_hub`
- **Result:** Succeeded initially
- **Problems:** `huggingface-cli` was later found to be deprecated:

```
Warning: 'huggingface-cli' is deprecated and no longer works. Use 'hf' instead.
```

- **Solution:** Use `hf download ...` instead of `huggingface-cli download ...`
- **Final status:** ☒ Resolved

Component: WhisperX (ASR model + package)

- **Purpose:** Speech-to-text for audio chunks
- **Installation method:** `pip install whisperx` (initially into main `.venv` — **this was a mistake**, later corrected)
- **Result of first attempt:** Installed, but caused major dependency conflicts (see Error 6 below)
- **Final solution:** Installed into a **separate, dedicated virtual environment** `.venv_whisperx`, matching a pattern the startup script (`start_online_asr_worker.sh`) already supported:

```
bash

python3 -m venv .venv_whisperx
source .venv_whisperx/bin/activate
pip install whisperx
deactivate
```

- **Final status:** ☒ Resolved — confirmed working (`whisperx OK: .../.venv_whisperx/lib/python3.10/site-packages/whisperx/__init__.py`), `torch: 2.8.0+cu128` isolated from main venv's `torch==2.12.0`)

Component: WhisperX ASR model weights (`faster-whisper-small`)

- **Purpose:** Actual speech recognition model

- **Source discovered:** `Systran/faster-whisper-small` on Hugging Face (Systran org publishes CTranslate2/faster-whisper conversions of Whisper models)
- **Command:**

```
bash

hf download Systran/faster-whisper-small --local-dir ~/models/whisperx
```

(Originally suggested `faster-whisper-large-v3`, but downgraded to `small` for CPU speed once no-GPU was confirmed.)

- **Problem:** Files were placed directly in `~/models/whisperx/` (flat), but the repo's model-resolution code expected them inside a `faster-whisper-{model_name}` subdirectory (`~/models/whisperx/faster-whisper-small/model.bin`).
- **Solution:**

```
bash

mkdir -p ~/models/whisperx/faster-whisper-small
mv ~/models/whisperx/config.json ~/models/whisperx/model.bin ~/models/whisperx/preprocessor_config.json ~/models/w
```

- **Final status:** ☒ Resolved

Component: WhisperX alignment model (English)

- **Purpose:** Word-level timestamp alignment for ASR
- **Source:** `facebook/wav2vec2-large-960h-lv60-self` (inferred from code's candidate name list: `{"en": ["wav2vec2-large-960h-lv60-self", "wav2vec2-base-960h"]}`)
- **Command:**

```
bash

hf download facebook/wav2vec2-large-960h-lv60-self --local-dir ~/models/whisperx/alignment/wav2vec2-large-960h-lv60-self
```

- **Result:** Downloaded and used successfully — log later confirmed:

```
Some weights of Wav2Vec2ForCTC were not initialized from the model checkpoint at
/home/tahagpt/models/whisperx/alignment/wav2vec2-large-960h-lv60-self and are newly initialized:
['wav2vec2.masked_spec_embed']
```

(This warning is expected/normal, not an error.)

- **Final status:** ☒ Resolved

Component: CLIP (visual embedding substitute)

- **Purpose:** Stand-in for VLM2Vec V2.0 (which requires a Qwen2-VL-2B backbone, impractical on CPU)
- **Source:** `openai/clip-vit-base-patch32`
- **Command:**

```
bash

mkdir -p ~/models/clip-vit-base-patch32
hf download openai/clip-vit-base-patch32 --local-dir ~/models/clip-vit-base-patch32
```

- **Result:** Downloaded successfully (~600MB, 398 weight files per load log)
- **Final status:** ☒ Resolved — confirmed producing 512-dim embeddings

Component: MiniLM (text embedding substitute)

- **Purpose:** Stand-in for Qwen3-Embedding-4B (too large for CPU)
- **Source:** `sentence-transformers/all-MiniLM-L6-v2`
- **Command:**

bash

```
pip install sentence-transformers
mkdir -p ~/models/minilm-embed
hf download sentence-transformers/all-MiniLM-L6-v2 --local-dir ~/models/minilm-embed
```

- **Result:** Downloaded successfully (~80MB, 103 weight files per load log)
- **Final status:** ☒ Resolved — confirmed producing 384-dim embeddings

Component: pandas

- **Purpose:** Required by an answer-generation code path (discovered via a `ModuleNotFoundError` deep in an `answer_debug` field)
- **Command:** `pip install pandas`
- **Result:** `Successfully installed pandas-2.3.3 pytz-2026.3.post1 tzdata-2026.3` — pinned version matched `requirements.txt` (`pandas==2.3.3`)
- **Final status:** ☒ Resolved

Component: Ollama

- **Purpose:** Local LLM serving (OpenAI-compatible API)
- **Installation method:** Already installed and running as a **systemd service** (`ollama.service`) before this was diagnosed — user had not started it manually.
- **Models pulled:**

bash

```
ollama pull llama3.2:3b
ollama pull llama3.2-vision # later found broken, see Error 14
ollama pull llava:7b        # working substitute
```

- **Version issue:** Initial version (`0.33.2`) failed to load (`llama3.2-vision`) with `error loading model: unknown model architecture: 'mllama'`. Web search confirmed this as an **active, open Ollama GitHub bug** (June 2026 issues #16547 and #16490 — a regression where a recent Ollama update broke `mllama` architecture support).
- **Update command given (not confirmed executed/re-verified after):**

bash

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl restart ollama
```

- **Final resolution:** Abandoned (`llama3.2-vision`) entirely in favor of (`llava:7b`), which uses a different (unaffected) architecture and was confirmed working.
- **Final status:** ☒ Resolved (via model substitution, not fixing the Ollama bug itself)

5. LLM / Model Setup

5.1 Models considered, in chronological order

Stage	Model(s) considered	Outcome
Initial	OpenAI GPT (<code>gpt-4o-mini</code> , <code>gpt-5.4</code>) etc., hardcoded default in repo)	Rejected — user has no OpenAI API key
2nd	Mistral API (<code>mistral-large-latest</code> , <code>mistral-small-latest</code>) via <code>OPENAI_BASE_URL=https://api.mistral.ai/v1</code>	Discussed as OpenAI-compatible option; not actually configured/tested
3rd	Groq API (<code>llama-3.3-70b-versatile</code> , vision: <code>meta-llama/llama-4-scout-17b-16e-instruct</code>) via <code>OPENAI_BASE_URL=https://api.groq.com/openai/v1</code>	Configured and partially tested working (see Section 15) — later abandoned once user clarified intent to run fully locally
Final	Ollama: <code>llama3.2:3b</code> (text), <code>llava:7b</code> (vision)	☒ Final working configuration

5.2 Local LLM quantity/size decision

User asked whether `(llama-3.3-70b-versatile)` (Groq's model) could be run locally. Assistant's analysis (based on general reasoning, not from the repo's docs) concluded 70B models are impractical on CPU (~35-40GB RAM for 4-bit quantization, ~1-2 tokens/sec). The repo's own `(.env.example)` "local" LLM profile (`(EM2MEM_LOCAL_LLM_MODEL_PATH=/path/to/Qwen3.5-9B)`) targets a 9B model and still assumes a GPU (`(EM2MEM_LOCAL_LLM_GPU_MEMORY_UTILIZATION=0.94)`, `(CUDA_VISIBLE_DEVICES=2)`), reinforcing that even the repo's own authors did not design for CPU-only local inference. **Decision:** proceed with a small model via Ollama (`(llama3.2:3b)`, later also `(llava:7b)` for vision) instead of the repo's `(local-qwen35)` profile.

5.3 Model-name plumbing bugs discovered (these are the most significant findings of the session)

Three separate, independent bugs in the repository's own source code were discovered and patched during this session — none were caused by the user's configuration:

1. `(src/em2mem/llm/llm_wrapper.py)` — `(_detect_provider())` only recognized model names containing the substring `( "gpt" )` or `( "qwen3" )`; anything else (`( llama-3.3-70b-versatile )`, `( llama3.2:3b )`, `( ollama/llama3.2:3b )`) raised `( ValueError: Unknown model name: {model_name} )`. This meant **every LLM call in the entire session, on both Groq and Ollama, was silently falling back to a hardcoded local string** until this was patched.
2. `(src/em2mem/llm/openai_gpt.py)` (line ~367) — A hardcoded `(MODEL_DICT)` allowlist (`(gpt-5)`, `(gpt-5.1)`...`(gpt-5.5)`, `(gpt-5-mini)`, `(gpt-5-nano)`, `(gpt-4o-mini)`) raised `( ValueError: Unsupported model: {model_name}. Available: [...] )` for anything not in the dict — blocking Groq and Ollama models even after bug #1 was fixed.
3. `(src/em2mem/llm/openai_gpt.py)` (line ~394) — `(self.model_name = MODEL_DICT[model_name])` would `( KeyError )` for any model not in the dict, even after the check at line 367 was patched to allow custom base URLs through.

All three were patched (see Section 7, Errors 11-13, for exact diffs).

5.4 Model-name format gotcha: LiteLLM-style prefixes do NOT work with Ollama directly

The user tried `(EM2MEM_QUERY_RESPOND_MODEL=ollama/llama3.2:3b)` (a LiteLLM routing convention). This **failed** against Ollama's native OpenAI-compatible endpoint:

```
NotFoundError: Error code: 404 - {'error': {'message': "model 'ollama/llama3.2:3b' not found", ...}}
```

A direct `(curl)` to Ollama confirmed the correct format has **no prefix**:

```
bash
curl -s http://127.0.0.1:11434/v1/chat/completions -d '{"model": "llama3.2:3b", ...}'
```

succeeded. **Lesson:** the `(provider/model)` prefix syntax is LiteLLM-specific and should not be used when the repo's code is calling Ollama's own OpenAI-compatible endpoint directly (not through LiteLLM's routing layer).

6. Server Setup

6.1 Starting the API server

```
bash
scripts/start_api.sh
```

- Reads `(.env)`, sets ffmpeg-related env vars, launches Uvicorn.
- Binds to `(EM2MEM_API_HOST=127.0.0.1)`, `(EM2MEM_API_PORT=8000)`.
- Logs to `(logs/api/server.log)`.
- Prints: `([start_api] API server pid=<pid> log=logs/api/server.log)`

If already running, prints: `([start_api] API server already running pid=<pid>; reusing it)` (this "already running" detection was later found to sometimes be a **false positive** based on stale runtime-state files rather than actually checking the live process — see Error 15).

6.2 Starting all workers

```
bash
```

```
scripts/start_online_all_workers.sh
```

Starts, in order: `preprocess`, `asr` (`voice_question`), `asr_stream` (`stream`), `stream`, `live_ingest`, `refine`, `consolidation`, `vlm2vec_embedding`, `visual`, `memory`, `query`. (Legacy `evidence` worker is disabled by default.)

Individual worker start scripts (used for targeted restarts):

```
bash

scripts/start_online_query_worker.sh
scripts/start_online_visual_worker.sh
scripts/start_online_asr_worker.sh --worker-name asr --task-filter voice_question --device cpu
scripts/start_online_mst_refine_worker.sh
scripts/start_online_memory_worker.sh
scripts/start_online_vlm2vec_embedding_server.sh
```

6.3 Standalone substitute embedding servers (custom-written, not part of the original repo)

Two custom FastAPI servers were written from scratch during this session to stand in for the repo's real embedding services, matching their exact HTTP contracts:

CLIP embedding server (`online_clip_embedding_server.py`), replacing the real VLM2Vec service:

```
bash

nohup python online_clip_embedding_server.py > clip_server.log 2>&1 &
disown
```

Listens on `127.0.0.1:18091`, implements `GET /health`, `POST /embed/images`, `POST /embed/texts`.

Text embedding server (`online_text_embedding_server.py`), replacing the real Qwen3 embedding service:

```
bash

nohup python online_text_embedding_server.py > text_embed_server.log 2>&1 &
disown
```

Listens on `127.0.0.1:18096`, implements `GET /health`, `POST /embed/texts`.

Both must be started and healthy **before** `scripts/start_online_all_workers.sh`, because `start_online_vlm2vec_embedding_server.sh` checks for an existing healthy service at the configured URL/port before deciding whether to launch the real (non-functional, offline) VLM2Vec service.

6.4 Checking whether the server is running

```
bash

curl http://127.0.0.1:8000/ping
# {"status":"ok","message":"Em2Mem API is running"}

curl http://127.0.0.1:18091/health
# {"status":"ok","model_loaded":true}

curl http://127.0.0.1:18096/health
# {"status":"ok","model_loaded":true}

python monitor_online_pipeline.py --watch
# (or: timeout 8 python monitor_online_pipeline.py --watch to auto-exit)
```

The monitor prints a table of all workers with `status` (`ready`/`stale`/`stopped`/`missing`/`failed`/`disabled`), `pid`, `age`, `model/client`, and `last_error`, plus queue depths per stage.

Other useful status endpoints discovered:

```
bash
```

```
curl http://127.0.0.1:8000/runtime
curl http://127.0.0.1:8000/pipeline_runtime
curl http://127.0.0.1:8000/stream/<session_id>/status
```

6.5 Stopping processes

No single unified "stop everything" script was found/used. The user/assistant used targeted `pkill` commands throughout:

```
bash
```

```
pkill -f "online_asr_worker.py"
pkill -f "online_query_worker.py"
pkill -9 -f "online_query_worker.py"    # forced kill, used when SIGTERM wasn't picked up in time
kill -9 <pid>                          # used once to kill a uvicorn process that didn't match the pkill pattern (s
```

A `scripts/stop_server_and_workers.sh` script exists in the repo's `scripts/` directory (seen in the directory listing) but was **not** used/tested in this conversation.

The query worker's own code (`online_query_worker.py`) was found to register `SIGTERM`/`SIGINT` handlers for graceful shutdown:

```
python
```

```
signal.signal(signal.SIGTERM, _handle_stop)
signal.signal(signal.SIGINT, _handle_stop)
```

confirming that `pkill` (`SIGTERM`) triggers a clean shutdown loop exit rather than an abrupt kill, printing `[query_worker] stopped` before exiting.

6.6 Restarting

Standard pattern used throughout the session:

```
bash
```

```
pkill -9 -f "online_query_worker.py"
scripts/start_online_query_worker.sh > logs/workers/query.log 2>&1 &
disown
sleep 8
tail -40 logs/workers/query.log
```

Lesson learned the hard way: background-launching a process and immediately `curl`-ing it in the same command block frequently raced ahead of the process finishing startup, producing false "connection refused" failures multiple times in this session. The fix each time was simply to re-run the health check after confirming (via `ps aux`) the process was actually alive.

7. Complete Error & Troubleshooting Log

Error 1 — `huggingface-cli` deprecated

Context: Attempting to download WhisperX model weights.

Command:

```
bash
```

```
huggingface-cli download Systran/faster-whisper-large-v3 --local-dir ~/models/whisperx
```

Error:

```
Warning: `huggingface-cli` is deprecated and no longer works. Use `hf` instead.
```

Diagnosis: The `huggingface_hub` package renamed its CLI entry point from `huggingface-cli` to `hf`.

Successful Solution:

```
bash
hf download Systran/faster-whisper-large-v3 --local-dir ~/models/whisperx
```

Final Status:  Resolved

Error 2 — CLIP (`get_image_features`) returns (`BaseModelOutputWithPooling`), not a tensor

Context: Testing a standalone CLIP script before integrating it as a VLM2Vec substitute.

Command:

```
python
image_embedding = model.get_image_features(**inputs)
print("Embedding shape:", image_embedding.shape)
```

Error:

```
AttributeError: 'BaseModelOutputWithPooling' object has no attribute 'shape'
```

Diagnosis: On (`transformers`) 5.x, (`get_image_features()`)/(`get_text_features()`) return a (`BaseModelOutputWithPooling`) object instead of a raw tensor (a breaking change from older (`transformers`) versions).

Successful Solution:

```
python
image_embedding = image_output.pooler_output if hasattr(image_output, "pooler_output") else image_output
```

Final Status:  Resolved

Error 3 — (`test_clip.py`) silently ran with no output, no error

Context: Re-running the (previously working) CLIP test script.

Command:

```
bash
python test_clip.py
```

Error: No error — just an empty prompt return, exit code 0.

Diagnosis: (`cat test_clip.py`) revealed the file was **empty** — content had apparently been lost/not saved during a prior copy/paste or file-write operation.

Successful Solution: Rewrote the file with (`cat > test_clip.py << 'EOF' ... EOF`).

Final Status:  Resolved

Error 4 — (`.env`) syntax error: unquoted placeholder value

Context: Running (`scripts/start_api.sh`) after editing (`.env`).

Command:

```
bash
scripts/start_api.sh
```

Error:

```
.env: line 129: syntax error near unexpected token `newline'
```

(Root cause identified on a **later** attempt, at a different line number, same underlying pattern:)

```
.env: line 76: syntax error near unexpected token `newline'
```

Diagnosis: `.env` is sourced with bash (`set -a; source .env`), so any value containing shell-special characters breaks parsing. The specific culprit found was:

```
EM2MEM_RUNTIME_ADMIN_TOKEN=<optional-admin-token>
```

Bash interprets `<` as input redirection, breaking the line.

Successful Solution:

```
bash
sed -i '/EM2MEM_RUNTIME_ADMIN_TOKEN=<optional-admin-token>/d' .env
```

(or set it to an empty value: `EM2MEM_RUNTIME_ADMIN_TOKEN=`)

General guidance also given: `grep -n '<.*>' .env` to find any other un-filled placeholder lines using the same pattern.

Final Status: ☒ Resolved

Error 5 — `EM2MEM_VLM2VEC_EMBED_URL` corrupted by copy-paste concatenation

Context: Recurred **multiple times** throughout the conversation.

Observed value:

```
EM2MEM_VLM2VEC_EMBED_URL=http://127.0.0.1:18091scripts/start_api.sh
```

Diagnosis: A copy-paste artifact accidentally appended `(scripts/start_api.sh)` onto the end of the URL value, likely from pasting terminal output/history back into the `.env` file. This caused the `(visual)` worker to fail with:

```
VLM2Vec embedding service is not ready at http://127.0.0.1:18091scripts/start_ap
```

Successful Solution (applied at least twice during the session, since it recurred):

```
bash
sed -i 's|EM2MEM_VLM2VEC_EMBED_URL=.*|EM2MEM_VLM2VEC_EMBED_URL=http://127.0.0.1:18091|' .env
```

Final Status: ☒ Resolved (each time it recurred) — flagged as a recurring risk worth double-checking after any bulk `.env` edit.

Error 6 — Installing `(whisperx)` into the main `.venv` broke the entire environment

Context: `(pip install whisperx)` was run directly inside the main `(.venv)` (not yet aware of the `(.venv_whisperx)` split pattern).

Command:

```
bash
pip install whisperx
```

Error (dependency conflicts reported by pip):

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This
behaviour is the source of the following dependency conflicts.
em2mem-online-server 0.1.0 requires numpy==2.0.2, but you have numpy 2.2.6 which is incompatible.
em2mem-online-server 0.1.0 requires transformers==5.10.2, but you have transformers 4.57.6 which is incompatible.
```

(whisperx) had also silently upgraded/replaced (torch), (numpy), (huggingface-hub), and (transformers), and pulled in a large, unrelated dependency tree (pyannote-audio, lightning, torch-audiomentations, optuna, etc.).

Diagnosis: WhisperX's dependency requirements ((numpy>=2.1.0), (transformers>=4.48.0)) resolving to a much newer/older version than pinned, (torch~=2.8.0) directly conflict with the repo's own pinned versions ((numpy==2.0.2), (transformers==5.10.2), (torch==2.12.0) per (pyproject.toml)/(requirements.txt)). This is exactly why the repository's own (DEPLOYMENT.md) documents a separate (.venv_whisperx) environment — a fact that was initially overlooked.

Attempts That Failed:

1. Attempted in-place repair: (pip uninstall whisperx faster-whisper ctranslate2 -y) then (pip install numpy==2.0.2 transformers==5.10.2) then (pip install -e .). This **partially worked** but then (pip install -e .) (via the (accelerate) dependency) pulled in an **unpinned** (torch==2.14.0) instead of the required (2.12.0), along with a new, incompatible CUDA 13.x toolchain, and dropped (torchaudio) entirely:

```
pyannote-audio 4.0.7 requires torchaudio>=2.8.0, which is not installed.
torch-audiomentations 0.12.0 requires torchaudio>=0.9.0, which is not installed.
torch-pitch-shift 1.2.5 requires torchaudio>=0.7.0, which is not installed.
```

Successful Solution: Full clean rebuild of the main (.venv):

```
bash

deactivate
cd ~/Documents/fyp/LightMem-Ego/src/backend
rm -rf .venv
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e .
pip install torch==2.12.0 torchvision==0.27.0 # explicit re-pin as a safety net
```

Confirmed via:

```
bash

python -c "import torch, torchvision; print(torch.__version__, torchvision.__version__)"
# 2.12.0+cu130 0.27.0+cu130
```

WhisperX was then installed **only** into a separate (.venv_whisperx):

```
bash

python3 -m venv .venv_whisperx
source .venv_whisperx/bin/activate
pip install whisperx
deactivate
```

Confirmed via:

```
bash
```

```
python -c "import whisperx; print('whisperx OK:', whisperx.__file__)"
# whisperx OK: .../.venv_whisperx/lib/python3.10/site-packages/whisperx/__init__.py
python -c "import torch; print('torch:', torch.__version__)"
# torch: 2.8.0+cu128
```

The `(scripts/start_online_asr_worker.sh)` script was confirmed to already contain logic preferring `(.venv_whisperx)` if present:

```
bash

if [[ -f ".venv_whisperx/bin/activate" ]]; then
    source ".venv_whisperx/bin/activate"
elif [[ -f ".venv/bin/activate" ]]; then
    source ".venv/bin/activate"
fi
```

Final Status: ☒ Resolved

Error 7 — `(asr_stream)` worker crash: `(IndexError: list index out of range)`

Context: Starting ASR workers after the `(.venv_whisperx)` split was completed.

Command: (via `(scripts/start_online_all_workers.sh)`)

Error:

```
File "online_preprocess/asr_whisperx.py", line 125, in _resolve_asr_model_name_or_path
    snapshot_root = candidates[2]
IndexError: list index out of range
```

Diagnosis: Two separate, layered causes:

1. **Directory-layout mismatch (the actual trigger):** The downloaded model files sat flat in `(~/models/whisperx/)`, but the function's model-resolution logic expected a `(faster-whisper-{model_name})` subdirectory. Because none of the earlier candidates matched, execution fell through to a buggy fallback branch.
2. **Genuine off-by-one bug in the repository's own code:** The `(candidates)` list in `(_resolve_asr_model_name_or_path())` was only ever populated with 2 items (indices 0 and 1), but the fallback code unconditionally tried `(candidates[2])`, which would always raise `(IndexError)` if reached, regardless of the user's file layout.

Attempts That Failed: None recorded — the directory-layout fix (Section 4, "WhisperX ASR model weights") was identified and applied directly.

Successful Solution (two parts):

Part A — fix the directory layout (see Section 4 for full commands):

```
bash

mkdir -p ~/models/whisperx/faster-whisper-small
mv ~/models/whisperx/{config.json,model.bin,preprocessor_config.json,README.md,tokenizer.json,vocabulary.json,vocabu
```

Part B — patch the genuine upstream bug (so it can never resurface even if this code path is hit again in future):

```
bash

sed -i 's/snapshot_root = candidates\[2\]/snapshot_root = candidates\[1\]/' online_preprocess/asr_whisperx.py
```

Reasoning given: `(candidates[1])` corresponds to `(root / f"models--Systran--{systran_name}" / "snapshots")`, which matches the surrounding comment describing a "Hugging Face snapshot-style cache" — the most likely originally-intended target. This was explicitly flagged as an inference, not a certainty, since the original author's intent could not be directly confirmed from the code alone.

Final Status:  Resolved (for this user's case, since execution now returns early via the `ct2_dir` check before ever reaching the patched line)

Error 8 — ASR workers launched with hardcoded `--device cuda` despite no GPU

Context: After Error 7 was fixed, ASR workers advanced further but then failed on model load.

Error:

```
RuntimeError: CUDA failed with error CUDA driver version is insufficient for CUDA runtime version
```

Diagnosis: `scripts/start_online_all_workers.sh` passed `--device "${EM2MEM_ASR_WHISPERX_DEVICE:-cuda}"` and `--device "${EM2MEM_STREAM_ASR_WHISPERX_DEVICE:-cuda}"` — i.e., these fall back to `cuda` **unless** specific env vars `(EM2MEM_ASR_WHISPERX_DEVICE)`, `(EM2MEM_STREAM_ASR_WHISPERX_DEVICE)` are set, which they were not. Setting the more generic `(EM2MEM_WHISPERX_DEVICE=cpu)` in `(.env)` had **no effect** on this specific code path, since the script reads different, more specific env var names.

Attempts That Failed:

1. `sed -i 's/--device cuda/--device cpu/g' scripts/start_online_all_workers.sh` — **did not match anything**, since the script uses variable interpolation `(--device "${VAR:-cuda}")`, not a literal string `--device cuda`.

Successful Solution: Set the correctly-named, more specific env vars in `(.env)`:

```
bash

cat >> .env << 'EOF'
EM2MEM_ASR_WHISPERX_DEVICE=cpu
EM2MEM_STREAM_ASR_WHISPERX_DEVICE=cpu
EOF
```

Final Status:  Resolved — confirmed via subsequent log line:

```
[asr_worker] WhisperX runtime preloaded
ASR worker ready: task_filter=stream stream_backend=whisperx audio_backend=whisperx voice_question_backend=xfyun
whisperx_loaded=True
```

Error 9 — Ollama port already in use / systemd service confusion

Context: Attempting to manually start Ollama.

Command:

```
bash

fuser -k 11434/tcp
ollama serve &
```

Error:

```
Error: listen tcp 127.0.0.1:11434: bind: address already in use
```

Diagnosis: Ollama was already running as a **systemd service** `(ollama.service)`, which auto-restarts on kill. `(fuser -k)` either killed a leftover process or briefly freed the port before systemd relaunched it.

Successful Solution: Confirmed via:

```
bash
```

```
systemctl status ollama
# ● ollama.service - Ollama Service ... Active: active (running)
curl http://127.0.0.1:11434/v1/models
# {"object": "list", "data": [{"id": "llama3.2:3b", ...}]}
```

No manual `ollama serve` needed — the systemd-managed instance was already correct. Guidance given for future management:

```
bash

sudo systemctl stop ollama
sudo systemctl disable ollama # prevent auto-start
sudo systemctl restart ollama # restart, instead of manual `ollama serve &`
```

Final Status:  Resolved

Error 10 — Text-caption model (`EM2MEM_VLM_MODEL`) defaults to `gpt-4o-mini`

Context: Investigating where the image-captioning model name is actually configured (not obviously named `EM2MEM_..._CAPTION_MODEL` as initially guessed).

Diagnosis (found via `grep`):

```
python

# online_preprocess/vlm_captioner.py, line 228
model=model or os.getenv("EM2MEM_VLM_MODEL", "gpt-4o-mini"),
```

Successful Solution (at the time, for the Groq phase):

```
EM2MEM_VLM_MODEL=meta-llama/llama-4-scout-17b-16e-instruct
```

(Groq's free vision-capable model, confirmed via web search to be available on Groq's free tier at the time, flagged as "Preview" status, not production-grade.)

Note: This var was **not** revisited/re-set once the project moved to Ollama; the pipeline's actual "image" LLM path in the final working test ended up going through `EM2MEM_QUERY_RESPOND_MODEL` (set to `llava:7b`), not `EM2MEM_VLM_MODEL` specifically — the exact relationship between these two variables in the live-answer path was not fully traced in this conversation.

Final Status:  Workaround used /  Partially resolved — value left in an intermediate (Groq-oriented) state; not confirmed relevant to the final Ollama-based working configuration.

Error 11 — `ValueError: Unknown model name` (provider auto-detection too strict)

Context: First successful end-to-end pipeline run, question-answering test returned the local fallback answer instead of an LLM-generated one.

Command: `POST /ask/{session_id}` → polled via `GET /query_task/{task_id}`

Error (found inside the task result's `answer_debug.error_debug` field):

```
ValueError: Unknown model name: llama-3.3-70b-versatile
```

(and later, after switching to Ollama:)

```
ValueError: Unknown model name: llama3.2:3b
```

Diagnosis: Found via `grep -rn "Unknown model name"`, pointing to `src/em2mem/llm/llm_wrapper.py:47`:

```
python
```

```

if "gpt" in model_name_lower:
    return "openai"
elif "qwen3" in model_name_lower:
    return "qwen3v1"
else:
    raise ValueError(f"Unknown model name: {model_name}")

```

Any model name not containing the literal substrings `"gpt"` or `"qwen3"` was rejected outright — affecting **both** Groq and Ollama model names, a bug entirely independent of which backend the user chose.

Successful Solution:

```

bash

python3 << 'EOF'
path = "/home/tahagpt/Documents/fyp/LightMem-Ego/src/backend/src/em2mem/llm/llm_wrapper.py"
with open(path, "r") as f:
    content = f.read()
old = '''        if model_name not in MODEL_DICT:
            raise ValueError(f"Unsupported model: {model_name}. Available: {list(MODEL_DICT.keys())}")'''
new = '''        _custom_base_url = os.getenv("OPENAI_BASE_URL")
            if model_name not in MODEL_DICT and not _custom_base_url:
                raise ValueError(f"Unsupported model: {model_name}. Available: {list(MODEL_DICT.keys())}")'''
content = content.replace(old, new)
with open(path, "w") as f:
    f.write(content)
EOF

```

(Note: this snippet actually corresponds to the **Error 12** patch below; the Error 11 fix itself simply changed the `else: raise ValueError(...)` branch in `_detect_provider()` to `return "openai"` instead, defaulting any non-`gpt`/non-`qwen3` model name to the generic OpenAI-compatible client, since that client already correctly uses `OPENAI_BASE_URL`/`OPENAI_API_KEY` for both Groq and Ollama.)

```

bash

sed -i 's/raise ValueError(f"Unknown model name: {model_name}")/return "openai" # default to OpenAI-compatible client/'

```

Final Status: ☒ Resolved (confirmed the specific error message stopped appearing after the patch; a **different, deeper** error then surfaced — see Error 12)

Error 12 — `ValueError: Unsupported model: ... Available: ['gpt-5', ...]` (hardcoded OpenAI-only allowlist)

Context: Immediately after fixing Error 11, running the same test again.

Error:

```

ValueError: Unsupported model: ollama/llama3.2:3b. Available: ['gpt-5', 'gpt-5.1', 'gpt-5.2', 'gpt-5.3', 'gpt-5.4', 'gpt-5.5', 'gpt-5-mini', 'gpt-5-nano', 'gpt-4o-mini']

```

Diagnosis: A **second**, independent hardcoded allowlist in `src/em2mem/llm/openai_gpt.py`:

```
python
```

```
MODEL_DICT = {
    "gpt-5": "gpt-5-2025-08-07",
    "gpt-5.1": "gpt-5.1",
    "gpt-5.2": "gpt-5.2",
    "gpt-5.3": "gpt-5.3",
    "gpt-5.4": "gpt-5.4",
    "gpt-5.5": "gpt-5.5",
    "gpt-5-mini": "gpt-5-mini-2025-08-07",
    "gpt-5-nano": "gpt-5-nano-2025-08-07",
    "gpt-4o-mini": "gpt-4o-mini",
}
...
if model_name not in MODEL_DICT:
    raise ValueError(f"Unsupported model: {model_name}. Available: {list(MODEL_DICT.keys())}")
```

This check runs **regardless** of whether a custom `OPENAI_BASE_URL` is set, meaning it blocks any non-OpenAI OpenAI-compatible backend (Groq, Ollama, etc.) categorically.

Successful Solution:

```
python

_custom_base_url = os.getenv("OPENAI_BASE_URL")
if model_name not in MODEL_DICT and not _custom_base_url:
    raise ValueError(f"Unsupported model: {model_name}. Available: {list(MODEL_DICT.keys())}")
```

Applied via a Python script that read the file, did a string replace, and wrote it back (see Section 5.3 for reasoning).

Final Status: ☒ Resolved for this specific check — but immediately revealed **Error 13** at the next line that actually uses `MODEL_DICT`.

Error 13 — `KeyError` risk from `MODEL_DICT[model_name]` (found proactively before it triggered)

Context: After patching Error 12, the assistant proactively checked for other usages of `MODEL_DICT` before retesting.

Command:

```
bash

grep -n "MODEL_DICT\[\" ~/Documents/fyp/LightMem-Ego/src/backend/src/em2mem/llm/openai_gpt.py
# 394:         self.model_name = MODEL_DICT[model_name]
```

Diagnosis: Even with the Error 12 check bypassed for custom base URLs, line 394 would still attempt a direct dictionary lookup and raise `KeyError` for any model name not in `MODEL_DICT`.

Successful Solution:

```
python

old = "        self.model_name = MODEL_DICT[model_name]"
new = "        self.model_name = MODEL_DICT.get(model_name, model_name)"
```

Applied via the same read/replace/write Python pattern.

Final Status: ☒ Resolved — confirmed via subsequent test runs advancing past this point entirely.

Error 14 — `ollama/llama3.2:3b` model name not found by Ollama itself (LiteLLM-prefix confusion)

Context: After Errors 11-13 were patched, the pipeline reached the actual HTTP call to Ollama, but still failed.

Error:

```
[QUERY-LLM-ERROR] path=chat.completions:image attempt=1/2 model=ollama/llama3.2:3b error=NotFoundError: Error
code: 404 - {'error': {'message': 'model 'ollama/llama3.2:3b' not found', 'type': 'not_found_error', 'param':
None, 'code': None}}
```

Diagnosis: The `ollama/` prefix is a **LiteLLM routing convention**, not something Ollama's own native OpenAI-compatible `/v1/chat/completions` endpoint understands. A direct test confirmed this:

```
bash

curl -s http://127.0.0.1:11434/v1/chat/completions -d '{"model": "llama3.2:3b", "messages": [...]}'
# succeeded, returned a real completion
```

vs. the model string with the prefix, which the pipeline was sending, failing.

Attempts That Failed:

1. `EM2MEM_QUERY_RESPOND_MODEL=ollama/llama3.2:3b` (and the same for `EM2MEM_MST_REFINE_MODEL`, `EM2MEM_MST_EPISODIC_MODEL`, `EM2MEM_MEMORY_MODEL`) — failed with the 404 above.

Successful Solution:

```
bash

sed -i 's|EM2MEM_QUERY_RESPOND_MODEL=.*|EM2MEM_QUERY_RESPOND_MODEL=llama3.2:3b|' .env
sed -i 's|EM2MEM_MST_REFINE_MODEL=.*|EM2MEM_MST_REFINE_MODEL=llama3.2:3b|' .env
sed -i 's|EM2MEM_MST_EPISODIC_MODEL=.*|EM2MEM_MST_EPISODIC_MODEL=llama3.2:3b|' .env
sed -i 's|EM2MEM_MEMORY_MODEL=.*|EM2MEM_MEMORY_MODEL=llama3.2:3b|' .env
```

(plain model name, no `ollama/` prefix)

Final Status:  Resolved

Error 15 — Query worker "already running" was a false positive (stale runtime-state file)

Context: After model-name fixes, `scripts/start_online_all_workers.sh` reported the query worker as already running and reused its (stale) PID, so config changes appeared to have no effect.

Command:

```
bash

scripts/start_online_all_workers.sh
# [start_online_all_workers] query already running runtime_pid=47455; reusing it
```

Diagnosis:

```
bash

ps aux | grep online_query_worker
# (no output — process was actually dead)
```

The startup script's "already running" check relies on a runtime-state file that can go stale if the process died without updating it, causing the script to skip relaunching a worker that is, in fact, not running at all.

Successful Solution: Launch the worker directly, bypassing the "already running" check in the aggregate script:

```
bash

scripts/start_online_query_worker.sh > logs/workers/query.log 2>&1 &
disown
```

Followed by explicit process verification (`ps aux | grep online_query_worker`) rather than trusting the aggregate script's own "already running" message.

Final Status: ☒ Resolved (as a workaround/practice — the underlying stale-state-detection bug in the startup script itself was **not** patched)

Error 16 — Multimodal request rejected by text-only model (`llama3.2:3b`)

Context: After all model-name plumbing bugs were fixed, the query pipeline attempted to attach image evidence (the uploaded test frame) to the LLM call, but the configured model was text-only.

Error:

```
[QUERY-LLM-ERROR] path=chat.completions:image attempt=1/2 model=llama3.2:3b error=BadRequestError: Error code: 400
- {'error': {'message': '{"error":{"code":400,"message":"Multimodal data provided, but model does not support
multimodal requests."},"type":"invalid_request_error"}}', ...}}
```

Diagnosis: `llama3.2:3b` has no vision capability. The pipeline's routing logic decided to attach image evidence (`"use_image_evidence_source": "auto_partial_memory_fallback"`) because the memory state was "partial/provisional" (the frame's MST event was still open/unclosed).

Attempts That Failed:

1. Passing (`"use_image_evidence": false`) in the (`/ask`) request body — **did not prevent** the image-attachment attempt; the same 400 error recurred. The conversation did not establish exactly why the flag was ignored (possibly overridden by the `auto_partial_memory_fallback` rule), and this was **not further investigated** — flagged as an open/unresolved sub-question.

Successful Solution: Switch to a vision-capable model. First attempted (`llama3.2-vision`) (see Error 17 for why that failed), ultimately succeeded with (`llava:7b`).

Final Status: ☒ Resolved (via model substitution) — the underlying question of why (`use_image_evidence: false`) didn't take effect remains **unresolved/not investigated**.

Error 17 — `llama3.2-vision` fails to load in Ollama: `unknown model architecture: 'mllama'`

Context: Attempting to use (`llama3.2-vision`) as the vision-capable model.

Command:

```
bash
ollama pull llama3.2-vision
# (then, via the pipeline:)
```

Error:

```
InternalServerError: Error code: 500 - {'error': {'message': "llama-server process has terminated: exit status 1:
error loading model: unknown model architecture: 'mllama'\nerror loading model: unknown model architecture:
'mllama'", ...}}
```

Diagnosis: Confirmed via web search to be a **known, currently-open Ollama bug** (GitHub issues #16547 and #16490, dated June 2026) — a regression in a recent Ollama release broke (`mllama`) architecture (used by `llama3.2-vision`) support entirely, affecting users who previously had it working. This is an upstream Ollama defect, not a local misconfiguration. (`ollama --version`) at the time of testing reported (`@.33.2`).

Attempts That Failed:

1. No workaround for (`llama3.2-vision`) itself was found or attempted (e.g., downgrading Ollama was not tried).

Successful Solution: Abandoned (`llama3.2-vision`); switched to (`llava:7b`), which uses a different (LLaVA) architecture unaffected by the (`mllama`) regression:

```
bash
ollama pull llava:7b
sed -i 's|EM2MEM_QUERY_RESPOND_MODEL=.*|EM2MEM_QUERY_RESPOND_MODEL=llava:7b|' .env
```

Confirmed working via direct test:

```
bash
ollama run llava:7b "describe this in one sentence"
```

and, more importantly, via the actual pipeline (see Section 15 for the final confirmed success).

An Ollama update was also suggested as a general remediation (`curl -fsSL https://ollama.com/install.sh | sh`) + (`sudo systemctl restart ollama`), but since `llava:7b` resolved the immediate blocker, it is unclear from the conversation whether the update was actually applied/needed.

Final Status:  Resolved (via model substitution) —  the underlying Ollama `ollama` bug itself remains **unresolved** (upstream issue, outside the user's control).

Error 18 — Query task stuck `in_progress` for a long time; suspected hang

Context: After switching to `llava:7b`, the first end-to-end test with image evidence took an extended time to complete, with the task status remaining `in_progress` well past initial expectations (checked after 20s, then 60s).

Diagnosis (investigated, not initially assumed):

- `top` showed no `ollama`/`llama-server` process at one check — but this coincided with a period where the API server itself had actually crashed and needed restarting (see Error 19), muddying initial diagnosis.
- After a clean full-stack restart, `systemctl status ollama` and `journalctl -u ollama` confirmed the `llama-server` child process (PID 6903) **was** actively working, logging real (if slow) progress:

```
decoding image batch 1/2, n_tokens_batch = 512
...
image decoded (batch 1/2) in 134507 ms # ~2.24 minutes for one image batch
decoding image batch 2/2, n_tokens_batch = 64
image decoded (batch 2/2) in 16288 ms
...
[GIN] ... | 200 | 14.173915766s | ... POST "/v1/chat/completions"
```

A separate direct request (14.17s) succeeded, and two earlier requests had actually timed out server-side at the 2-minute mark (`| 500 | 2m0s |`) purely because the repo's HTTP client's retry/timeout window was shorter than the image-decoding time on this CPU-only hardware.

- `free -h` at one point showed the system under real memory pressure: only 179.6 MiB free RAM, 3.5GiB of 3.7GiB swap in use, with multiple heavy VS Code processes also competing for RAM.

Diagnosis (final): Not a hang or crash — genuinely slow CPU-bound image inference (multi-minute image encoding), compounded by low available system memory. **The exact cause of the two earlier 500/timeout responses was established** (client-side timeout shorter than server processing time); the conversation did not fully confirm whether memory pressure alone would have caused an outright failure if given more time, so that specific causal link is presented as a contributing factor, not a proven root cause.

Successful Solution: Patience — waiting long enough (a full clean restart, then re-issuing the request, then polling with generous `sleep` intervals) allowed a request to eventually succeed within the client's retry window once the model's KV-cache/prompt was "warm" from a previous attempt (`cached n_tokens = 1238`), completing in 14.17s the second/third time around versus multiple minutes cold.

Final Status:  Resolved (the actual final query succeeded) —  Note that first-time (cold) vision-model inference on this hardware can take well over 2 minutes, longer than the pipeline's built-in retry timeout, which may cause intermittent apparent failures on a system under memory pressure. This is a **performance limitation**, not a bug, and is flagged as a known constraint for future work (see Section 17).

Error 19 — API server died silently mid-session

Context: After extensive worker restart cycles, a `/stream/start` request returned completely empty output.

Command:

```
bash
```

```
curl -s -X POST http://127.0.0.1:8000/stream/start -d '{"input_mode": "frame_audio_stream"}' > /tmp/session_start.js
python3 -c "import json; print(json.load(open('/tmp/session_start.json'))['session_id'])"
```

Error:

```
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

(i.e., the response file was completely empty)

Diagnosis: The API server process had died at some point during the session (not definitively traced to a specific cause in the conversation), leaving nothing listening on port 8000.

Successful Solution:

```
bash
scripts/start_api.sh
sleep 3
curl -s http://127.0.0.1:8000/ping
```

which relaunched it successfully (`[start_api] API server pid=4312 log=logs/api/server.log`).

Final Status: ☒ Resolved — root cause of the original crash was **not established** in the conversation.

Error 20 — `(pkill)` pattern didn't match a `(uvicorn)`-launched API process

Context: Attempting a full clean shutdown of all processes before a fresh restart.

Command:

```
bash
pkill -9 -f "api_server.py"
ps aux | grep -E "online_api_server" | grep -v grep
```

Error (unexpected leftover process):

```
tahagpt 4312 ... python -m uvicorn api_server:app --host 127.0.0.1 --port 8000
```

Diagnosis: This particular instance of the API server had been started via `(python -m uvicorn api_server:app ...)` rather than a script invoking `(api_server.py)` directly, so the process's command-line string didn't contain the literal substring `(api_server.py)`, and the `(pkill -f "api_server.py")` pattern failed to match it.

Successful Solution:

```
bash
kill -9 4312
```

(direct PID-based kill, found by manually inspecting `(ps aux)` output rather than relying solely on the pattern-based `(pkill)`)

Final Status: ☒ Resolved

Error 21 — Stale 22.2-hour-old worker records reported as "stale" by monitor

Context: After the API server crash/restart (Error 19) and Error 20, `(monitor_online_pipeline.py --watch)` showed all main workers as `(status: stale)`, `(age: 22.2h)`, and the query worker as `(status: stopped)`.

Diagnosis: The runtime-state files tracking worker health had not been updated in 22.2 hours, meaning the actual worker processes referenced were long dead (consistent with them having been killed at various points throughout the very long session), while the monitor was still reporting their last-known (stale) state.

Successful Solution: A full, careful, ordered restart of the entire stack from scratch, with explicit `sleep` waits and health checks at each stage (embedding servers → API server → all workers), rather than relying on any component's "already running" self-report:

```
bash

# (see Section 6 / Section 14 for the complete, exact sequence used)
```

Final Status: ✔ Resolved — confirmed via a subsequent clean `monitor_online_pipeline.py --watch` output showing all workers `ready`.

8. Failed Approaches

Approach	Why It Was Tried	Result	Why Abandoned
<code>EM2MEM_QUERY_RESPOND_MODEL=ollama/llama3.2:3b</code> (LiteLLM-style prefix)	Assumed the repo used LiteLLM's provider-routing convention	<code>NotFoundError: model 'ollama/llama3.2:3b' not found</code> from Ollama itself	Ollama's native OpenAI-compatible endpoint doesn't understand the <code>provider/</code> prefix; the repo calls Ollama's endpoint directly, not through LiteLLM's router
<code>sed -i 's/--device cuda/--device cpu/g' scripts/start_online_all_workers.sh</code>	Attempted to force CPU mode for ASR workers	No matches found, no effect	The script used variable interpolation (<code>--device "\${VAR:-cuda}"</code>), not a literal string, so the specific env vars (<code>EM2MEM_ASR_WHISPERX_DEVICE</code> , <code>EM2MEM_STREAM_ASR_WHISPERX_DEVICE</code>) had to be set instead
In-place repair of the main <code>.venv</code> after the WhisperX contamination (<code>pip uninstall</code> + reinstall pinned versions)	Tried to avoid a full <code>venv</code> rebuild to save time	Partially worked, but a subsequent <code>pip install -e .</code> pulled in an unpinned <code>torch==2.14.0</code> via the <code>accelerate</code> dependency, reintroducing conflicts	The transitive dependency resolution was unpredictable; a full clean <code>rm -rf .venv</code> rebuild was more reliable
<code>llama3.2-vision</code> as the vision-capable model	Natural first choice for local vision LLM via Ollama, matching the <code>llama3.2:3b</code> text model family	Failed to load: <code>unknown model architecture: 'mllama'</code>	Confirmed as an active upstream Ollama bug (GitHub issues #16547, #16490); switched to <code>llava:7b</code> instead
Passing <code>"use_image_evidence": false</code> in the <code>/ask</code> request to avoid the multimodal 400 error before a vision model was configured	Quick way to test text-only Q&A in isolation	Image evidence was still attached; same 400 error recurred	The exact override logic was not investigated further in this conversation; documented as an open question
<code>curl</code> immediately after backgrounding a server process in the same command block (recurred multiple times, e.g. CLIP server, text-embedding server, API server)	Convenience — combining launch and verification in one command block	Frequently returned "connection refused" even though the server was actually starting/running fine	Race condition between the background process finishing startup and the <code>curl</code> firing; resolved each time by simply re-running the health check after confirming via <code>ps aux</code> that the process was alive

9. Decisions & Choices Made

Decision: Use `openai/clip-vit-base-patch32` (CLIP) as a substitute for VLM2Vec V2.0 for visual embeddings. **Reason:** VLM2Vec V2.0 is built on a Qwen2-VL-2B backbone with a custom pooling head; no official quantized/GGUF version exists (confirmed via web search), and running the full model on CPU was assessed as impractically slow. CLIP is ~600MB, well-supported on CPU, and produces usable image/text embeddings via a standard interface. **Alternatives considered:** bitsandbytes int8/int4 quantization of the VLM2Vec backbone (rejected — CUDA-only, no CPU benefit); ONNX + dynamic INT8 quantization (rejected — would require custom export code for VLM2Vec's non-standard forward pass); skipping visual embeddings entirely (offered as an option, not chosen). **Final choice:** CLIP, wrapped in a custom FastAPI server matching the real VLM2Vec HTTP contract (`/health`, `/embed/images`, `/embed/texts`).

Decision: Use `sentence-transformers/all-MiniLM-L6-v2` as a substitute for Qwen3-Embedding-4B for text embeddings. **Reason:** Qwen3-Embedding-4B is a 4B-parameter model, assessed as too heavy for the CPU-only setup, and no local weights were found on the system. **Alternatives considered:** None explicitly discussed besides the direct substitution approach already validated for the visual embedding case. **Final choice:** MiniLM (~80MB), wrapped in a custom FastAPI server matching the real Qwen3 text-embedding HTTP contract (`/health`, `/embed/texts`).

Decision: Isolate WhisperX into a separate virtual environment (`.venv_whisperx`) rather than the main `.venv`. **Reason:** WhisperX's dependency requirements directly conflicted with the repository's pinned dependency versions, and the repository's own `start_online_asr_worker.sh` script and `DEPLOYMENT.md` documentation already anticipated and supported this exact separation pattern. **Alternatives considered:** Attempting to reconcile version conflicts within a single shared `.venv` — tried, but caused a cascading series of dependency issues (see Error 6), ultimately abandoned. **Final choice:** Dedicated `.venv_whisperx`, confirmed compatible with the existing worker-launch scripts' auto-detection logic.

Decision: Use the WhisperX `small` model size instead of `large-v3`. **Reason:** No GPU was available; `small` offers acceptable accuracy for testing at meaningfully faster CPU inference speed than `large-v3`. **Alternatives considered:** `large-v3` (originally suggested first, before the no-GPU constraint was confirmed), `medium`, `base`. **Final choice:** `small`.

Decision: Move from Groq (cloud API) to Ollama (fully local) for LLM inference. **Reason:** The user explicitly clarified partway through the session: "we are not calling groq, we are running everything locally btw" — indicating a firm requirement for a fully local/offline setup, overriding the earlier Groq-based configuration that had been set up (and was, unbeknownst to both parties at the time, not even actually working due to the then-undiscovered `llm_wrapper.py` bug). **Alternatives considered:** Groq (`llama-3.3-70b-versatile` text, `meta-llama/llama-4-scout-17b-16e-instruct` vision) — configured and explored first; Mistral API — discussed but never actually configured. **Final choice:** Ollama, `llama3.2:3b` (text) and `llava:7b` (vision).

Decision: Use `llava:7b` instead of `llama3.2-vision` for the vision-capable LLM. **Reason:** `llama3.2-vision` failed to load in the installed Ollama version due to a confirmed upstream Ollama bug (`unknown model architecture: 'mllama'`). **Alternatives considered:** Updating Ollama via the official install script (suggested, not confirmed as tested/applied); `qwen2.5vl:7b` (suggested as an alternative, not actually pulled/tested in this conversation). **Final choice:** `llava:7b`, confirmed working end-to-end.

Decision: Add `EM2MEM_ASR_WHISPERX_DEVICE=cpu` and `EM2MEM_STREAM_ASR_WHISPERX_DEVICE=cpu` to `.env`, rather than editing `scripts/start_online_all_workers.sh` directly. **Reason:** Keeping the fix in `.env` avoids it being silently overwritten by a future `git pull` of the repository, and is more maintainable/discoverable than a script edit. **Alternatives considered:** Directly editing the script's `--device cuda` default (attempted first via `sed`), but failed since the string wasn't a literal match — see Error 8/Failed Approaches). **Final choice:** `.env`-based override using the correctly-named, more specific environment variables.

10. Configuration Reference

10.1 Final known `.env` state (values as last confirmed in the conversation; some fields' final state is uncertain due to the volume

of edits — see notes)

Variable	Final Known Value	Notes
EM2MEM_API_HOST	127.0.0.1	
EM2MEM_API_PORT	8000	
EM2MEM_CORS_ORIGINS	http://localhost:5173,http://127.0.0.1:5173	
EM2MEM_PIPELINE_MODE	mst	
EM2MEM_AUTO_PREPROCESS	1	
EM2MEM_AUTO_MST_CONSOLIDATION	1	
EM2MEM_AUTO_MEMORY	1	
EM2MEM_AUTO_VISUAL_EMBEDDING	1	
EM2MEM_STREAM_ASR_ENABLED	1	
EM2MEM_STREAM_ASR_BACKEND	whisperx	
EM2MEM_AUDIO_ASR_ENABLED	1	
EM2MEM_AUDIO_ASR_BACKEND	whisperx	
EM2MEM_FFMPEG_BIN	ffmpeg	
EM2MEM_FFPROBE_BIN	ffprobe	
EM2MEM_WHISPERX_MODEL	small	Changed from initial medium
EM2MEM_WHISPERX_DEVICE	cpu	Changed from initial cuda; later found insufficient alone — see EM2MEM_ASR_WHISPERX_DEVICE
EM2MEM_ASR_WHISPERX_DEVICE	cpu	Added later — the actually-effective var for the asr worker
EM2MEM_STREAM_ASR_WHISPERX_DEVICE	cpu	Added later — the actually-effective var for the asr_stream worker
EM2MEM_WHISPERX_COMPUTE_TYPE	int8	Changed from initial float16 (not well-supported on CPU)
EM2MEM_WHISPERX_MODEL_DIR	/home/tahagpt/models/whisperx	
EM2MEM_WHISPERX_ALIGN_MODEL_DIR	/home/tahagpt/models/whisperx/alignment	Was a literal /path/to/whisperx/alignment placeholder initially
EM2MEM_WHISPERX_ALIGN_LANGS	zh,en	Not changed; zh retained even though only English use was implied — not explicitly revisited
EM2MEM_VISUAL_BACKEND	remote	
EM2MEM_VLM2VEC_MODEL_PATH	/home/tahagpt/models/clip-vit-base-patch32	Points at the CLIP substitute, not real VLM2Vec
EM2MEM_VLM2VEC_DEVICE	cpu	Changed from initial cuda
EM2MEM_VLM2VEC_DTYPE	float32	Changed from initial float16 (not well-supported on CPU); not actually

Variable

Final Known Value

Notes

consumed by the custom CLIP server,
but kept for consistency

`EM2MEM_VISUAL_BATCH_SIZE``8``EM2MEM_VLM2VEC_EMBED_HOST``127.0.0.1``EM2MEM_VLM2VEC_EMBED_PORT``18091``EM2MEM_VLM2VEC_EMBED_URL``http://127.0.0.1:18091`

Repeatedly got corrupted to
`...18091scripts/start_api.sh`
via copy-paste; fixed multiple times

`EM2MEM_ALLOW_HF_DOWNLOAD``0``EM2MEM_TEXT_EMBED_BACKEND``remote`

Added later (not in original
`.env.example` shown)

`EM2MEM_TEXT_EMBED_URL``http://127.0.0.1:18096`

Points at the MiniLM substitute server

`EM2MEM_VLM_MODEL``meta-llama/llama-4-scout-17b-16e-instruct`

Set during the Groq phase; **not confirmed updated** after the move to Ollama/llava

`EM2MEM_MEMORY_MODEL``llama3.2:3b`

Went through `gpt-5.4` → `llama-3.3-70b-versatile` → `llama3.2:3b` → briefly `ollama/llama3.2:3b` (failed) → back to `llama3.2:3b`

`EM2MEM_QUERY_RESPOND_MODEL``llava:7b`

Went through `gpt-5.4` → `llama-3.3-70b-versatile` → `llama3.2:3b` → `ollama/llama3.2:3b` (failed) → `llama3.2:3b` → `llama3.2-vision` (failed, Ollama bug) → `llava:7b` (final, working)

`EM2MEM_MST_REFINE_MODEL``llama3.2:3b`

Same history as
`EM2MEM_MEMORY_MODEL`

`EM2MEM_MST_EPISODIC_MODEL``llama3.2:3b`

Same history as
`EM2MEM_MEMORY_MODEL`

`EM2MEM_MST_REFINE_MAX_ATTEMPTS``3`

Unchanged

`EM2MEM_MST_REFINE_RETRY_BACKOFF_SECONDS``300`

Unchanged

`EM2MEM_EVIDENCE_CAPTION_BACKEND``openai`

Unchanged (this refers to "OpenAI-compatible protocol," not literally OpenAI)

`EM2MEM_ANSWER_LANGUAGE``en`

Changed from initial `zh`

`EM2MEM_VOICE_QUESTION_INITIAL_PROMPT``(removed/unset)`

Was a Mandarin-Chinese ASR priming string in the original template; removed since English was the target language

`EM2MEM_CURRENT_FAST_PATH_ENABLED``1`

Unchanged

`EM2MEM_RECENT_ST_*` (several threshold vars)`Unchanged from template`

See original `.env.example` dump for exact values

Variable	Final Known Value	Notes
OPENAI_API_KEY	ollama (final)	Went through <your-key> → <your-mistral-key> → <your-groq-key> → ollama
OPENAI_BASE_URL	http://127.0.0.1:11434/v1 (final)	Went through unset → https://api.mistral.ai/v1 (discussed, not confirmed set) → https://api.groq.com/openai/v1 → http://127.0.0.1:11434/v1
EM2MEM_LLM_PROFILE	remote	Unchanged throughout — "remote" here means "use the OPENAI_* vars," regardless of whether that's OpenAI, Groq, or Ollama
EM2MEM_LLM_PROFILE_OVERRIDE	(empty)	Unchanged
EM2MEM_RUNTIME_ADMIN_TOKEN	(removed/commented out)	Was the literal placeholder <optional-admin-token>, which broke .env parsing; removed
EM2MEM_LIVE_RTMP_ENABLED	0	Unchanged — not used in this conversation
EM2MEM_WEBRTC_WHIP_ENABLED	0	Unchanged — not used in this conversation
EM2MEM_SINGLE_ACTIVE_SESSION	0	Unchanged

Note on EM2MEM_LOCAL_LLM_* block: Left entirely unused/untouched, since EM2MEM_LLM_PROFILE=remote throughout meant these variables (EM2MEM_LOCAL_LLM_MODEL_PATH=/path/to/Qwen3.5-9B, etc.) were never actually read by the running configuration.

10.2 Ports summary

Port	Service
8000	Main API server (api_server.py) / uvicorn
18091	CLIP embedding server (online_clip_embedding_server.py) — substitute for VLM2Vec
18096	Text embedding server (online_text_embedding_server.py) — substitute for Qwen3 embeddings
11434	Ollama (systemd service)
5173	Frontend dev server (referenced in CORS config; frontend not actually run in this conversation)

10.3 Model paths

Path	Contents
~/models/whisperx/faster-whisper-small/	WhisperX ASR model (model.bin) + tokenizer/config files
~/models/whisperx/alignment/wav2vec2-large-960h-lv60-self/	WhisperX English alignment model
~/models/clip-vit-base-patch32/	CLIP model (VLM2Vec substitute)
~/models/minilm-embed/	all-MiniLM-L6-v2 model (Qwen3-Embedding substitute)
~/models/VLM2Vec-V2.0/	Real VLM2Vec-V2.0 weights, downloaded early in the conversation but never actually used once the CLIP substitute path was chosen

11. Directory / File Structure

Based on the `find` directory listing obtained during the conversation, the relevant parts of `src/backend/` are:

text

```

src/backend/
├── api_server.py                # Main FastAPI app
├── online_worker.py
├── online_stream_worker.py
├── online_live_ingest_worker.py
├── online_asr_worker.py
├── online_visual_worker.py
├── online_memory_worker.py
├── online_query_worker.py
├── online_mst_refine_worker.py
├── online_mst_consolidation_worker.py
├── online_evidence_worker.py    # legacy, disabled by default
├── online_vlm2vec_embedding_server.py # real VLM2Vec service (not used – CLIP substitute used instead)
├── online_qwen3_embedding_server.py # real Qwen3 embedding service (not used – MiniLM substitute used instead)
├── online_clip_embedding_server.py # CUSTOM – written during this session
├── online_text_embedding_server.py # CUSTOM – written during this session
├── monitor_online_pipeline.py    # CLI pipeline health monitor
├── .env / .env.example
├── pyproject.toml
├── requirements.txt
├── requirements-asr.txt
├── .venv/                      # main environment (pinned versions)
├── .venv_whisperx/            # isolated WhisperX environment
├── clip_server.log             # CUSTOM log file
├── text_embed_server.log       # CUSTOM log file
├── logs/
│   ├── api/server.log
│   └── workers/{asr,asr_stream,consolidation,live_ingest,memory,preprocess,query,refine,stream,visual,vlm2vec_embedding}
├── online_tasks/               # on-disk task queues (query/, etc.)
├── online_sessions/<session_id>/ # per-session data (frames, audio, memory)
├── docs/
│   ├── online_stream_api_contract.md
│   └── ask_streaming_frontend_contract.md
├── scripts/
│   ├── start_api.sh
│   ├── start_online_all_workers.sh
│   ├── start_online_query_worker.sh
│   ├── start_online_visual_worker.sh
│   ├── start_online_asr_worker.sh
│   ├── start_online_mst_refine_worker.sh
│   ├── start_online_mst_consolidation_worker.sh
│   ├── start_online_memory_worker.sh
│   ├── start_online_vlm2vec_embedding_server.sh
│   ├── start_online_stream_worker.sh
│   ├── start_online_live_ingest_worker.sh
│   ├── start_online_qwen3_embedding_server.sh
│   ├── stop_server_and_workers.sh # exists in repo; not used in this conversation
│   └── (many other utility/test scripts)
├── online_current/, online_short_term/, online_memory_incremental/, online_visual/, online_preprocess/, online_pipeline/ # internal pipeline subpackages
├── src/
│   ├── em2mem/                # the actual "em2mem" package (installed editable)
│   │   ├── llm/
│   │   │   ├── llm_wrapper.py # PATCHED (Error 11)
│   │   │   ├── openai_gpt.py  # PATCHED (Errors 12, 13)
│   │   │   ├── qwen3vl.py
│   │   │   └── templates/
│   │   ├── memory/
│   │   │   ├── episodic/, semantic/, visual/
│   │   │   └── EM2Memory.py
│   │   └── embedding/
│   │       └── embedding_wrapper.py

```

```
├── vlm2vecv2.py           # real VLM2Vec wrapper (not used at runtime)
├── qwen3_embedding.py     # real Qwen3 wrapper (not used at runtime)
├── remote_text_embedding.py # HTTP client the CLIP/MiniLM substitutes had to match
├── VLM2Vec/              # vendored VLM2Vec source (Qwen2-VL backbone code)
└── HippoRAG/             # vendored HippoRAG source (long-term retrieval)
```

Note: `em2mem_online_server.egg-info/` in the backend root confirmed that `em2mem` is not a separate external package but is built and installed in editable mode from `src/em2mem/` as part of the same repository.

12. Dependencies

Python packages (main `.venv`, pinned per `requirements.txt`/`pyproject.toml`)

Dependency	Version	Purpose	Required?	Notes
torch	<code>2.12.0</code>	ML runtime	Yes	Repeatedly got overwritten by unpinned versions via transitive deps (<code>2.13.0</code> , <code>2.14.0</code>) — required explicit re-pinning
torchvision	<code>0.27.0</code>	ML runtime	Yes	
numpy	<code>2.0.2</code>	Numerical computing	Yes	Conflicted with WhisperX's <code>numpy>=2.1.0</code> requirement
transformers	<code>5.10.2</code>	Model loading (CLIP, VLM2Vec backbone, etc.)	Yes	Conflicted with WhisperX's <code>transformers>=4.48.0</code> resolving to incompatible versions
pandas	<code>2.3.3</code>	Used somewhere in the answer-generation path	Yes	Missing initially, causing a <code>ModuleNotFoundError</code> deep in an LLM call
fastapi	<code>0.136.3</code>	Web framework (main API + custom substitute servers)	Yes	
uvicorn	<code>0.49.0</code>	ASGI server	Yes	
sentence-transformers	(latest available at install time, <code>5.5.1</code> per one log)	Text embedding substitute (MiniLM)	Yes (added)	
huggingface_hub	(varies — see below)	Model downloads	Yes	Version churned repeatedly due to conflicting sub-dependency requirements from different packages
litellm	<code>1.88.1</code>	Present in dependency list; not confirmed as the actual LLM call path — see Section 5.4 note on prefix confusion	Unclear	The <code>ollama/</code> prefix confusion suggested LiteLLM might be involved, but the actual code inspected (<code>llm_wrapper.py</code> , <code>openai_gpt.py</code>) used a raw <code>openai</code> -SDK-style client, not <code>litellm.completion()</code> directly

Isolated `.venv_whisperx` packages

Dependency	Version	Purpose
whisperx	<code>3.8.6</code>	ASR
torch	<code>2.8.0+cu128</code>	WhisperX's own torch requirement, deliberately isolated from the main <code>2.12.0</code>
faster-whisper	<code>1.2.1</code>	WhisperX's transcription backend
ctranslate2	<code>4.8.2</code>	faster-whisper's inference engine
pyannote-audio, lightning, torch-audiomentations, optuna, etc.	(various)	Pulled in transitively by whisperx; not directly used by the pipeline but required for import

External services / tools

Tool	Purpose	Required?
Ollama (systemd service)	Local LLM serving	Yes (final architecture)
ffmpeg / ffprobe	Audio/video processing	Yes
Docker (<code>ossrs/srs:5</code> image referenced in <code>.env</code>)	Live RTMP streaming (SRS server)	No — not used in this conversation (<code>EM2MEM_LIVE_RTMP_ENABLED=0</code>)

Models (Hugging Face downloads)

Model	Source	Size (approx, as stated)	Used in final config?
<code>Systran/faster-whisper-small</code>	Hugging Face	Unknown / not specified in chat	✔ Yes
<code>facebook/wav2vec2-large-960h-lv60-self</code>	Hugging Face	Unknown / not specified in chat	✔ Yes
<code>openai/clip-vit-base-patch32</code>	Hugging Face	~600MB	✔ Yes (VLM2Vec substitute)
<code>sentence-transformers/all-MiniLM-L6-v2</code>	Hugging Face	~80MB	✔ Yes (Qwen3-Embedding substitute)
<code>VLM2Vec/VLM2Vec-V2.0</code>	Hugging Face	Multi-GB (exact size not confirmed)	✗ No — downloaded but abandoned in favor of CLIP

13. Working Setup

Environment

- Ubuntu, CPU-only, no GPU
- Python 3.10, two separate venvs: `.venv` (main, pinned deps) and `.venv_whisperx` (isolated WhisperX deps)

Model

- Text LLM: `llama3.2:3b` (Ollama)
- Vision LLM: `llava:7b` (Ollama)
- Visual embeddings: CLIP (`openai/clip-vit-base-patch32`)
- Text embeddings: MiniLM (`sentence-transformers/all-MiniLM-L6-v2`)
- ASR: WhisperX + `Systran/faster-whisper-small`

Runtime

- Ollama running as a systemd service on port `11434`
- Two custom FastAPI embedding servers on ports `18091` (CLIP) and `18096` (MiniLM)
- Main API server (FastAPI/uvicorn) on port `8000`
- Nine background workers (stream, live_ingest, preprocess, refine, consolidation, visual, memory, query, plus dedicated asr/asr_stream workers)

Server

See Section 6 for full startup/shutdown details.

Configuration

See Section 10 for the complete `.env` reference.

Startup sequence (as actually used at the point of final confirmed success)

```
bash
```

```

cd ~/Documents/fyp/LightMem-Ego/src/backend
source .venv/bin/activate

# 1. Embedding substitute servers
nohup python online_clip_embedding_server.py > clip_server.log 2>&1 &
disown
nohup python online_text_embedding_server.py > text_embed_server.log 2>&1 &
disown
sleep 10
curl -s http://127.0.0.1:18091/health
curl -s http://127.0.0.1:18096/health

# 2. API server
scripts/start_api.sh
sleep 5
curl -s http://127.0.0.1:8000/ping

# 3. All workers
scripts/start_online_all_workers.sh
sleep 20

# 4. Verify
timeout 8 python monitor_online_pipeline.py --watch

```

Verification (the exact test that produced a confirmed correct answer)

```

bash

# Create a session
curl -s -X POST http://127.0.0.1:8000/stream/start \
  -H "Content-Type: application/json" \
  -d '{"input_mode": "frame_audio_stream"}' > /tmp/session_start.json
python3 -c "import json; print(json.load(open('/tmp/session_start.json'))['session_id'])" > /tmp/session_id.txt
export SESSION_ID=$(cat /tmp/session_id.txt)

# Upload a test frame
python3 -c "from PIL import Image; Image.new('RGB', (640,480), 'green').save('/tmp/frame_001.jpg')"
curl -s -X POST http://127.0.0.1:8000/stream/$SESSION_ID/frame \
  -F "frame=@/tmp/frame_001.jpg" \
  -F "frame_index=1" \
  -F "relative_ts_ms=1000" \
  -F "format=jpg"

# Ask a question
curl -s -X POST http://127.0.0.1:8000/ask/$SESSION_ID \
  -H "Content-Type: application/json" \
  -d '{"question": "What did you just see?", "response_mode": "legacy", "mode": "async", "memory_mode": "auto"}' > /tmp/ask_result.json
python3 -c "import json; print(json.load(open('/tmp/ask_result.json'))['task_id'])" > /tmp/task_id.txt

# Poll for the result (allow 2+ minutes for vision inference on CPU)
sleep 120
curl -s http://127.0.0.1:8000/query_task/$(cat /tmp/task_id.txt) | python3 -c "
import json,sys
d = json.load(sys.stdin)
print('status:', d.get('status'))
print('Answer:', d['result'].get('answer'))
print('Fallback used:', d['result'].get('raw', {}).get('answer_debug', {}).get('fallback_used'))
"

```

Confirmed final output:

```
status: done
Answer: The image displays a grey background with a green rectangular overlay that seems to obscure part of the scene.
Fallback used: False
Error debug: None
```

This correctly describes the actual solid-green test frame that was uploaded, confirming the vision model genuinely processed the real image rather than hallucinating or falling back to a templated string.

Shutdown

bash

```
pkill -9 -f "online_asr_worker.py"
pkill -9 -f "online_visual_worker.py"
pkill -9 -f "online_vlm2vec_embedding_server.py"
pkill -9 -f "online_clip_embedding_server.py"
pkill -9 -f "online_text_embedding_server.py"
pkill -9 -f "online_memory_worker.py"
pkill -9 -f "online_mst_consolidation_worker.py"
pkill -9 -f "online_mst_refine_worker.py"
pkill -9 -f "online_stream_worker.py"
pkill -9 -f "online_live_ingest_worker.py"
pkill -9 -f "online_query_worker.py"
pkill -9 -f "online_worker.py"
pkill -9 -f "api_server.py"
# Also check for any process not matching these patterns (e.g. launched via `python -m uvicorn ...`):
ps aux | grep -E "online_|api_server|uvicorn" | grep -v grep
# and kill any leftover PID directly: kill -9 <pid>
```

Ollama itself was left running as a systemd service (not stopped between sessions):

bash

```
sudo systemctl stop ollama    # only if needed
```

14. Reproduction Guide

text

Step 1 – Prepare environment

```
git clone https://github.com/zjunlp/LightMem-Ego.git
cd LightMem-Ego/src/backend
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e .
pip install torch==2.12.0 torchvision==0.27.0 # explicit re-pin, safety net against transitive upgrades
```

Step 2 – Install WhisperX in an ISOLATED venv (do not install into the main .venv)

```
python3 -m venv .venv_whisperx
source .venv_whisperx/bin/activate
pip install whisperx
deactivate
```

Step 3 – Download and place ASR model

```
hf download Systran/faster-whisper-small --local-dir ~/models/whisperx
mkdir -p ~/models/whisperx/faster-whisper-small
mv ~/models/whisperx/{config.json,model.bin,preprocessor_config.json,README.md,tokenizer.json,vocabulary.json,vocab.json} ~/models/whisperx/faster-whisper-small
hf download facebook/wav2vec2-large-960h-lv60-self --local-dir ~/models/whisperx/alignment/wav2vec2-large-960h-lv60-self
```

Step 4 – Download visual/text embedding substitutes

```
mkdir -p ~/models/clip-vit-base-patch32
hf download openai/clip-vit-base-patch32 --local-dir ~/models/clip-vit-base-patch32
pip install sentence-transformers
mkdir -p ~/models/minilm-embed
hf download sentence-transformers/all-MiniLM-L6-v2 --local-dir ~/models/minilm-embed
```

Step 5 – Install and configure Ollama (or confirm systemd service already running)

```
systemctl status ollama # if not active: curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3.2:3b
ollama pull llava:7b # NOT llama3.2-vision – known broken in Ollama 0.33.2 (mllama architecture bug)
```

Step 6 – Create the two custom embedding-substitute server files

(see Section 15 / conversation for full source of `online_clip_embedding_server.py` and `online_text_embedding_server.py`)

```
pip install fastapi uvicorn
```

Step 7 – Patch three upstream bugs in the repository source

- `src/em2mem/llm/llm_wrapper.py` – change the final ``else: raise ValueError(...)`` in `_detect_provider()` to ``return "openai"``
- `src/em2mem/llm/openai_gpt.py` (~line 367) – guard the `MODEL_DICT` check with ``and not os.getenv("OPENAI_BASE_URL")``
- `src/em2mem/llm/openai_gpt.py` (~line 394) – change ``self.model_name = MODEL_DICT[model_name]`` to ``self.model_name = MODEL_DICT.get(model_name, model_name)``
- (Optional, for robustness) `online_preprocess/asr_whisperx.py` – change ``snapshot_root = candidates[2]`` to ``snapshot_root = candidates[1]``

Step 8 – Configure .env

```
cp .env.example .env
# Set (at minimum):
EM2MEM_WHISPERX_MODEL=small
EM2MEM_WHISPERX_DEVICE=cpu
EM2MEM_ASR_WHISPERX_DEVICE=cpu
EM2MEM_STREAM_ASR_WHISPERX_DEVICE=cpu
EM2MEM_WHISPERX_COMPUTE_TYPE=int8
EM2MEM_WHISPERX_MODEL_DIR=/home/<user>/models/whisperx
EM2MEM_WHISPERX_ALIGN_MODEL_DIR=/home/<user>/models/whisperx/alignment
EM2MEM_VISUAL_BACKEND=remote
EM2MEM_VLM2VEC_MODEL_PATH=/home/<user>/models/clip-vit-base-patch32
EM2MEM_VLM2VEC_DEVICE=cpu
EM2MEM_VLM2VEC_EMBED_URL=http://127.0.0.1:18091
```

```
EM2MEM_TEXT_EMBED_BACKEND=remote
EM2MEM_TEXT_EMBED_URL=http://127.0.0.1:18096
EM2MEM_MEMORY_MODEL=llama3.2:3b
EM2MEM_QUERY_RESPOND_MODEL=llava:7b
EM2MEM_MST_REFINE_MODEL=llama3.2:3b
EM2MEM_MST_EPISODIC_MODEL=llama3.2:3b
OPENAI_API_KEY=ollama
OPENAI_BASE_URL=http://127.0.0.1:11434/v1
EM2MEM_LLM_PROFILE=remote
# Remove/comment out any literal <placeholder> lines, especially EM2MEM_RUNTIME_ADMIN_TOKEN
```

Step 9 – Start embedding substitute servers

```
nohup python online_clip_embedding_server.py > clip_server.log 2>&1 &
disown
nohup python online_text_embedding_server.py > text_embed_server.log 2>&1 &
disown
sleep 10
curl http://127.0.0.1:18091/health
curl http://127.0.0.1:18096/health
```

Step 10 – Start API server

```
scripts/start_api.sh
sleep 5
curl http://127.0.0.1:8000/ping
```

Step 11 – Start all workers

```
scripts/start_online_all_workers.sh
sleep 20
timeout 8 python monitor_online_pipeline.py --watch
```

Step 12 – Run a test

(see Section 13, "Verification", for the exact session/frame/ask/query_task sequence)

Step 13 – Stop everything

(see Section 13, "Shutdown")

15. Verification & Testing

Health checks used throughout

```
bash

curl http://127.0.0.1:8000/ping
curl http://127.0.0.1:18091/health
curl http://127.0.0.1:18096/health
curl http://127.0.0.1:11434/v1/models
timeout 8 python monitor_online_pipeline.py --watch
```

CLIP embedding test (standalone, before pipeline integration)

```
python
```

```

from transformers import CLIPModel, CLIPProcessor
from PIL import Image
import torch
model = CLIPModel.from_pretrained(model_path)
processor = CLIPProcessor.from_pretrained(model_path)
image = Image.new("RGB", (224, 224), color="red")
inputs = processor(images=image, return_tensors="pt")
with torch.no_grad():
    image_output = model.get_image_features(**inputs)
    image_embedding = image_output.pooler_output if hasattr(image_output, "pooler_output") else image_output
print("Embedding shape:", image_embedding.shape)

```

Result: `torch.Size([1, 512])` for both image and text embeddings — confirmed working.

CLIP substitute server test (through the actual HTTP contract)

```

bash

curl -X POST http://127.0.0.1:18091/embed/images \
-H "Content-Type: application/json" \
-d '{"image_paths": ["/tmp/test.jpg"], "normalize": true}'

```

Result: Returned a 512-dimensional embedding array — confirmed working.

Text embedding substitute server test

```

bash

curl -X POST http://127.0.0.1:18096/embed/texts \
-H "Content-Type: application/json" \
-d '{"texts": ["a person walking in a park"], "normalize": true}'

```

Result: Returned a 384-dimensional embedding array — confirmed working.

Direct Ollama tests (bypassing the pipeline, used to isolate whether failures were in Ollama or the repo's client code)

```

bash

curl -s http://127.0.0.1:11434/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model": "llama3.2:3b", "messages": [{"role": "user", "content": "Say hello in one word."}]}'
# {"choices":[{"message":{"content":"Hi."}}, ...] -- succeeded

ollama run llama:7b "describe this in one sentence"
# " A person is standing in front of a building that has a clock on it."
# (a hallucinated answer, since no actual image was passed to this CLI invocation — used only to confirm the model 1

```

Full end-to-end pipeline test (final, successful)

See Section 13, "Verification," for the exact commands and confirmed output:

```

status: done
Answer: The image displays a grey background with a green rectangular overlay that seems to obscure part of the
scene.
Fallback used: False
Error debug: None

```

Earlier (Groq-based) partial test — for historical record

Before the pivot to Ollama, a similar end-to-end test against Groq's `llama-3.3-70b-versatile` was attempted. This actually returned `"status": "done"` and appeared successful at first, but was **later discovered to have silently used the local fallback answer the entire time** (`fallback_used: true`), (`error_debug: ValueError: Unknown model name: llama-3.3-70b-versatile`) due to Error 11 (`llm_wrapper.py`)'s provider-detection bug), which had not yet been found/patched at that point in the conversation.

This is an important caveat: **the pipeline can return `"status": "done"` with a plausible-looking answer even when the LLM call failed and a fallback was silently used** — always check `answer_debug.fallback_used` and `answer_debug.error_debug`, not just the top-level `status` field, when verifying a real LLM response.

16. Performance Observations

- **RAM:** System has 15832.2 MiB total RAM. At one point during heavy load (multiple VS Code windows + pipeline processes), free RAM dropped to 179.6 MiB with 3.5GiB of swap in use.
 - **Vision LLM inference (CPU, `llava:7b` via Ollama):** Per Ollama's own logs (`journalctl -u ollama`):
 - Image batch 1/2 decoding: `134507 ms` (~2 minutes 15 seconds) on a cold/first request
 - Image batch 2/2 decoding: `16288 ms` (~16 seconds)
 - A subsequent (warm-cache) request completed in `14.173915766s` total, with `prompt eval time = 583.66 ms / 1 tokens` and `eval time = 13547.02 ms / 25 tokens (~1.77 tokens per second)`
 - Two earlier requests **timed out** at exactly the 2-minute mark (`[ 500 | 2m0s ]`), consistent with the repo's HTTP client having a shorter timeout/retry window than the cold-start image-decoding time on this hardware.
 - **Text-only LLM inference (CPU, `llama3.2:3b` via Ollama):** One observed generation took `generation_ms=2620` (2.62 seconds) once the model call was actually succeeding (versus `generation_ms=2` for the near-instant local fallback string, a useful signal for distinguishing real LLM calls from fallbacks).
 - **Context length:** Not explicitly discussed/configured in this conversation beyond the repo's default `EM2MEM_LOCAL_LLM_MAX_MODEL_LEN=16384` (unused, since the local-qwen35 profile was never activated).
 - **Quantization:** Not explicitly configured for the Ollama models (Ollama's default quantization for `llama3.2:3b` and `llava:7b` pulls was used as-is; not discussed in detail).
-

17. Known Problems & Limitations

Problem	Status	Workaround	Permanent Solution?
<code>preprocess</code> worker's ASR backend shows as <code>xfyun</code> , not <code>whisperx</code>	⚠️ Unresolved / flagged but not fixed	None applied	Unknown — would need investigation into whether <code>xfyun</code> credentials are required or whether this worker's backend can be forced to <code>whisperx</code>
<code>asr</code> (<code>voice_question</code>) worker uses <code>xfyun</code> backend, sits idle with <code>whisperx_loaded=False</code>	⚠️ Unresolved / flagged but not fixed	None — not blocking, since voice-question ASR wasn't exercised in the final tests	Unknown
<code>"use_image_evidence": false</code> request parameter does not actually suppress image-evidence attachment	❌ Unresolved	None found	Unknown — needs further investigation into the <code>auto_partial_memory_fallback</code> override logic
<code>EM2MEM_VLM_MODEL</code> left set to a Groq model name (<code>meta-llama/llama-4-scout-17b-16e-instruct</code>) after moving to Ollama	⚠️ Unresolved / potentially stale config	None applied	Should be updated to an Ollama vision model or verified whether this variable is actually still consulted anywhere in the final working path
<code>llama3.2-vision</code> cannot be used with the installed Ollama version	❌ Unresolved (upstream bug)	Switched to <code>llava:7b</code> instead	Depends on an upstream Ollama fix for the <code>mlama</code> architecture regression (GitHub issues #16547, #16490)
Cold-start vision-model inference (2+ minutes) can exceed the pipeline's built-in HTTP client retry/timeout window, causing intermittent apparent failures	⚠️ Partially mitigated (works once "warm")	Retrying / waiting longer	Would need either a longer client-side timeout configured in the repo, or a persistently "warm" Ollama model (e.g., via <code>ollama run</code> kept alive, or Ollama's <code>keep_alive</code> setting — not discussed in this conversation)
<code>scripts/start_online_all_workers.sh</code> 's "already running" detection can report false positives based on stale runtime-state files	✅ Worked around	Launch specific worker scripts directly and verify with <code>ps aux</code> instead of trusting the aggregate script's self-report	The underlying stale-state-detection logic in the script itself was not patched
Root cause of one mid-session API server crash was never established	❌ Unresolved	Restarted the server	Unknown — recommend monitoring <code>logs/api/server.log</code> more closely in future sessions if it recurs
System memory pressure (as low as 179.6 MiB free) observed during heavy concurrent load (VS Code + full pipeline)	⚠️ Noted, not resolved	None applied within this conversation	Closing unnecessary applications (e.g., extra VS Code windows) before running the pipeline would likely help, though not explicitly tested

18. Current State

Is the system currently working? Yes — the last confirmed test in the conversation produced a correct, non-fallback, LLM-generated answer (`(Fallback used: False)`, `(Error debug: None)`) that accurately described the actual uploaded test image.

What components are working?

- API server (`/ping`), `/stream/start`, `/stream/{id}/frame`, `/ask/{id}`, `/query_task/{id}`)
- CLIP-based visual embedding substitute (port 18091)
- MiniLM-based text embedding substitute (port 18096)
- WhisperX ASR (isolated venv, CPU, confirmed loading and marked `whisperx_loaded=True` for the `stream` ASR task filter)

- Ollama-served (llama3.2:3b) (text) and (llava:7b) (vision) LLM inference
- All 9 pipeline workers reporting (ready) in (monitor_online_pipeline.py)
- Full end-to-end flow: session creation → frame upload → memory event creation → question asking → real vision-grounded LLM answer

What components are broken / unconfirmed?

- (preprocess) and (asr) (voice_question) workers still show (xfyun) as their backend rather than (whisperx) — not investigated/fixed
- ("use_image_evidence": false) request flag does not behave as expected
- (EM2MEM_VLM_MODEL) may hold a stale Groq-oriented value not applicable to the current Ollama-based setup
- The exact cause of one API-server crash mid-session was never determined

What commands should be used? See Section 13 (Working Setup) and Section 6 (Server Setup) for the exact, confirmed-working startup/shutdown/verification commands.

What remains to be done?

- Investigate and fix the (preprocess)/(asr) worker's (xfyun) backend default if (whisperx)-based preprocessing/voice-question handling is required
- Investigate why (use_image_evidence: false) doesn't suppress image attachment
- Reconcile/update (EM2MEM_VLM_MODEL) for the Ollama-based setup
- Test with a real (non-solid-color) photo and/or real spoken audio to exercise ASR and more realistic visual captioning
- Consider whether Ollama's (keep_alive) setting or a longer client timeout would reduce intermittent cold-start failures on vision queries
- Consider documenting/automating the full startup sequence into a single script, given how many manual steps and ordering dependencies were required

What should be avoided?

- Installing (whisperx) (or any package with conflicting pinned dependencies) directly into the main (.venv)
- Using LiteLLM-style (provider/model) prefixes (e.g., (ollama/llama3.2:3b)) when calling Ollama's native OpenAI-compatible endpoint directly
- Assuming a (curl) immediately following a backgrounded process launch will succeed — always verify via (ps aux) and/or add a real (sleep)
- Trusting (scripts/start_online_all_workers.sh)'s "already running" messages without independently verifying via (ps aux)
- Attempting (llama3.2-vision) on the currently-installed Ollama version ((0.33.2)) due to the confirmed (mllama) architecture bug

Next logical step (as suggested at the end of the conversation, not yet acted upon): Either (a) write up this deployment process as documentation (this report), (b) test with a more realistic photo/audio input, or (c) resolve the (preprocess) worker's (xfyun)-vs-(whisperx) discrepancy.

19. Chronological Timeline

text

(No absolute dates were consistently available for the entire conversation; relative ordering and the few timestamps that did appear in tool output/logs are used below.)

Phase 1 – Initial repo orientation

- |— Asked for exact guide to run LightMem-Ego
- |— Backend/frontend/glasses-app setup instructions provided (git clone, venv, .env.example, scripts/start_api.sh, s
- |— Uploaded excerpt of .env showing OPENAI_API_KEY / OPENAI_BASE_URL placeholders
- |— Asked about using Mistral API instead of OpenAI – confirmed OpenAI-compatible endpoint exists
- |— Asked about using Groq API instead – confirmed OpenAI-compatible endpoint, model-name caveats raised

Phase 2 – Model acquisition planning

- |— Asked where to get EM2MEM_WHISPERX_MODEL_DIR / EM2MEM_VLM2VEC_MODEL_PATH models
- |— Explained WhisperX (faster-whisper/Systran) and VLM2Vec concepts
- |— Provided download commands (initially huggingface-cli, later corrected to `hf`)
- |— User confirmed: Ubuntu machine, no GPU

Phase 3 – CPU-only pivot

- |— Discussed quantization options for VLM2Vec (bitsandbytes CUDA-only, ONNX custom work) – both rejected
- |— Decision: use CLIP (openai/clip-vit-base-patch32) as a lightweight visual-embedding substitute
- |— CLIP test script written, hit Error 2 (BaseModelOutputWithPooling), fixed
- |— CLIP test script silently empty (Error 3), fixed
- |— Asked about free vision models for captioning – Groq's meta-llama/llama-4-scout-17b-16e-instruct suggested

Phase 4 – Real repo source inspection begins

- |— User provided actual repo file tree (find output) – revealed src/em2mem/, online_visual/vlm2vec_runtime.py, embed
- |— Confirmed EM2MEM_VISUAL_BACKEND=remote architecture: main backend calls a standalone HTTP embedding service, not
- |— Decision: write a custom FastAPI server (online_clip_embedding_server.py) matching the real service's HTTP contr

Phase 5 – First .env corruption issues

- |— .env line 76/129 syntax error – traced to EM2MEM_RUNTIME_ADMIN_TOKEN=<optional-admin-token> – fixed
- |— EM2MEM_VLM2VEC_EMBED_URL corrupted with "scripts/start_api.sh" suffix (first occurrence) – fixed

Phase 6 – API server and CLIP server brought up

- |— scripts/start_api.sh run, confirmed via /ping
- |— online_clip_embedding_server.py launched, confirmed via /health and /embed/images

Phase 7 – Worker startup and cascading errors

- |— scripts/start_online_all_workers.sh run
- |— asr_stream worker: ModuleNotFoundError: No module named 'whisperx' (whisperx never installed) – Error triggers f

Phase 8 – WhisperX installation crisis

- |— pip install whisperx into main .venv – succeeds but causes massive dependency conflicts (Error 6)
- |— asr_stream then fails differently: IndexError in _resolve_asr_model_name_or_path (Error 7)
- |— Directory-layout fix applied (faster-whisper-small subdirectory)
- |— Genuine off-by-one bug found and patched (candidates[2] -> candidates[1])
- |— ASR then fails with CUDA error despite EM2MEM_WHISPERX_DEVICE=cpu (Error 8)
- |— Root cause found: scripts/start_online_all_workers.sh uses different, more specific env var names
- |— EM2MEM_ASR_WHISPERX_DEVICE / EM2MEM_STREAM_ASR_WHISPERX_DEVICE added – ASR finally loads successfully

Phase 9 – Full venv rebuild

- |— Main .venv found still contaminated by earlier whisperx install
- |— Attempted incremental repair – pip install -e . reintroduces unpinned torch==2.14.0
- |— Full rm -rf .venv + rebuild performed; torch/torchvision explicitly re-pinned
- |— whisperx re-installed cleanly into separate .venv_whisperx – confirmed isolated and working

Phase 10 – Missing Qwen3 text-embedding service discovered

- |— query_worker fails: RuntimeError, Qwen3 text embedding service not ready at :18096
- |— remote_text_embedding.py source inspected – same HTTP-contract pattern as VLM2Vec
- |— Decision: MiniLM (all-MiniLM-L6-v2) as substitute, same custom-FastAPI-server approach
- |— online_text_embedding_server.py written and confirmed working

Phase 11 – First full-pipeline end-to-end test (session e9e2e417a730's predecessor, session 43a0d4b1b9fc)

```
|— /stream/start, /stream/{id}/frame, /ask/{id}, /query_task/{id} all exercised successfully at the HTTP level
|— Answer returned, but later found to be a silent fallback (fallback_used: true) – real bug hunt begins

Phase 12 – LLM plumbing bug hunt (the deepest technical portion of the session)
|— Error 10: EM2MEM_VLM_MODEL default (gpt-4o-mini) found via grep
|— Error 11: llm_wrapper.py "Unknown model name" ValueError – patched (defaults unknown models to "openai" provider)
|— Error 12: openai_gpt.py MODEL_DICT allowlist ValueError – patched (bypass when custom OPENAI_BASE_URL set)
|— Error 13: MODEL_DICT[model_name] KeyError risk – proactively found and patched (.get() with fallback)
|— User clarifies: "we are not calling groq, we are running everything locally" – pivot to Ollama confirmed
|— Model names updated from llama-3.3-70b-versatile (Groq) to llama3.2:3b (Ollama)
|— Error 14: ollama/llama3.2:3b prefix rejected by Ollama itself (404) – LiteLLM-prefix confusion diagnosed and fixed

Phase 13 – Vision-model saga
|— Text-only llama3.2:3b succeeds for a plain text query (generation_ms=2620, confirmed real LLM call)
|— Same model then fails on an image-attached query: "Multimodal data provided, but model does not support multimodal"
|— ollama pull llama3.2-vision attempted
|— Error 17: llama3.2-vision fails to load – "unknown model architecture: 'mllama'" – confirmed via web search as a typo
|— Decision: switch to llava:7b instead

Phase 14 – Infrastructure instability and full clean restart
|— $SESSION_ID lost across shell command blocks multiple times – switched to file-based (/tmp/session_id.txt) persistence
|— /tmp cleared unexpectedly (system event, cause unconfirmed) – session state lost, had to recreate
|— API server found dead (Error 19) – restarted
|— Leftover uvicorn process not matched by pkill pattern (Error 20) – killed by explicit PID
|— monitor_online_pipeline.py shows all workers "stale" (22.2h old) / query "stopped" (Error 21)
|— Full, careful, ordered clean restart performed: embedding servers -> API -> all workers, with real sleep waits and status checks
|— Confirmed all workers "ready"

Phase 15 – Final successful test
|— New session e9e2e417a730 created; test frame (solid green image) uploaded
|— Query submitted, initially appears stuck "in_progress" for over a minute
|— Diagnosed via `top`, `free -h`, `systemctl status ollama`, `journalctl -u ollama`: not stuck, just slow (multi-tenant)
|— Task eventually completes: "The image displays a grey background with a green rectangular overlay..."
|— Fallback used: False, Error debug: None – CONFIRMED FULLY WORKING, FULLY LOCAL, END-TO-END PIPELINE

Phase 16 – Report requested
|— User requests this comprehensive technical report (current document)
```

20. Command Reference

Installation

```
bash

git clone https://github.com/zjunlp/LightMem-Ego.git
cd LightMem-Ego/src/backend
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e .
pip install torch==2.12.0 torchvision==0.27.0
pip install pandas
pip install fastapi uvicorn sentence-transformers

# Isolated WhisperX environment (DO NOT install into main .venv)
python3 -m venv .venv_whisperx
source .venv_whisperx/bin/activate
pip install whisperx
deactivate
```

Model downloads

```
bash

hf download Systran/faster-whisper-small --local-dir ~/models/whisperx
hf download facebook/wav2vec2-large-960h-lv60-self --local-dir ~/models/whisperx/alignment/wav2vec2-large-960h-lv60-self
hf download openai/clip-vit-base-patch32 --local-dir ~/models/clip-vit-base-patch32
hf download sentence-transformers/all-MiniLM-L6-v2 --local-dir ~/models/minilm-embed

ollama pull llama3.2:3b
ollama pull llava:7b
```

Environment

```
bash

cp .env.example .env
grep -n "_MODEL=" .env
sed -i 's|EM2MEM_QUERY_RESPOND_MODEL=.*|EM2MEM_QUERY_RESPOND_MODEL=llava:7b|' .env
```

Start server / workers

```
bash

scripts/start_api.sh
nohup python online_clip_embedding_server.py > clip_server.log 2>&1 &
nohup python online_text_embedding_server.py > text_embed_server.log 2>&1 &
scripts/start_online_all_workers.sh
scripts/start_online_query_worker.sh > logs/workers/query.log 2>&1 &
```

Check server

```
bash

curl http://127.0.0.1:8000/ping
curl http://127.0.0.1:18091/health
curl http://127.0.0.1:18096/health
curl http://127.0.0.1:11434/v1/models
timeout 8 python monitor_online_pipeline.py --watch
ps aux | grep -E "online_api_server|uvicorn" | grep -v grep
```

Stop server

```
bash

pkill -9 -f "online_query_worker.py"
pkill -9 -f "api_server.py"
kill -9 <pid> # for processes not matching pkill patterns, e.g. `python -m uvicorn`
```

Restart

```
bash

pkill -9 -f "online_query_worker.py"
scripts/start_online_query_worker.sh > logs/workers/query.log 2>&1 &
disown
sleep 8
```

Logs

```
bash
```

```
tail -40 logs/workers/query.log
tail -50 logs/api/server.log
cat clip_server.log
cat text_embed_server.log
sudo journalctl -u ollama -n 50 --no-pager
```

Troubleshooting

```
bash

systemctl status ollama
free -h
top -bn1 | head -15
sudo lsof -i :18091
grep -rn "Unknown model name" src/em2mem/llm/*.py
grep -n "EM2MEM_VLM2VEC_EMBED_URL" .env
```

21. Lessons Learned

What was actually learned from this chat:

- The repository's `DEPLOYMENT.md` documentation exists precisely because WhisperX's dependencies genuinely conflict with the main package's pinned versions — this was independently re-confirmed the hard way rather than being followed proactively from the start.
- LiteLLM's `provider/model` prefix convention (e.g., `ollama/llama3.2:3b`) does not apply when a codebase calls a provider's native OpenAI-compatible endpoint directly rather than routing through LiteLLM itself.
- A model name auto-detection function that only checks for specific substrings (`"gpt"`, `"qwen3"`) is fragile and will silently reject any other valid, working model name — this pattern was found twice in the same codebase (`llm_wrapper.py` and `openai_gpt.py`), suggesting it's worth grepping broadly for similar hardcoded allowlists before assuming a first fix is complete.
- A pipeline returning HTTP `"status": "done"` does not guarantee the underlying LLM call actually succeeded — the local/template fallback answer can look superficially plausible; always check dedicated debug fields (`answer_debug.fallback_used`, `answer_debug.error_debug`) to confirm.
- Backgrounding a process and immediately health-checking it in the same shell command block is unreliable; a real `sleep` plus a `ps aux` check is more trustworthy than a script's own "already running" self-report.
- Vision-model (multimodal) inference on CPU-only hardware can take well over two minutes on a cold start, which can exceed a client's built-in retry timeout and produce misleading intermittent failures that look like bugs but are actually a performance/timeout mismatch.
- Not all model-loading errors are local misconfiguration — the `llama3.2-vision`/`mllama` failure was confirmed (via web search of the Ollama GitHub repository) to be a genuine, currently-open upstream bug affecting many users, not something fixable from the client side.

General recommendations/inferences (not directly confirmed in the conversation):

- Consider using Ollama's `keep_alive` parameter to keep vision models warm between requests, reducing repeated cold-start penalties (not tested in this conversation).
- Consider wrapping the entire startup sequence (embedding servers → API → workers, each gated by a real health-check poll loop rather than a fixed `sleep`) into a single idempotent shell script to reduce the manual, error-prone, multi-step process observed throughout this session.

22. Recommendations for Future Work

Based strictly on what was discussed/observed in this conversation:

1. Investigate the `xfyun` ASR backend default for the `preprocess` and `asr` (voice_question) workers — determine whether this is intentional, requires credentials the user doesn't have, or can/should be redirected to `whisperx` like the `asr_stream` worker.
2. Investigate the `use_image_evidence: false` override behavior — the request parameter did not suppress image attachment as expected; this was flagged but not root-caused.

3. **Reconcile** (`EM2MEM_VLM_MODEL`) — currently still set to a Groq model name from an earlier phase; verify whether this variable is consulted anywhere in the current Ollama-based working path, and update or remove it accordingly.
 4. **Test with realistic inputs** — the successful end-to-end test used a synthetic solid-green test image and silent (zero-amplitude) audio; testing with a real photograph and real recorded speech would better validate the ASR and vision-captioning quality.
 5. **Address the cold-start timeout mismatch** for vision-model queries — either configure a longer client-side timeout in the repository's LLM-calling code, or find a way to keep the Ollama vision model "warm" (this recommendation is the assistant's own inference, not something the conversation confirmed a fix for).
 6. **Automate the startup sequence** — given how many discrete, order-dependent manual steps were required (and how often stale process/state issues caused confusion), a single startup script with built-in health-check polling (rather than fixed `sleep` durations) would likely improve reliability for future sessions.
 7. **Document the two upstream-bug patches** (`llm_wrapper.py`, `openai_gpt.py`) clearly in project notes or as a maintained patch/diff file, since a future `git pull` of the upstream repository would silently revert them, reintroducing the same failures.
 8. **Consider submitting the discovered bugs upstream** (the `candidates[2]` `IndexError`, the provider-detection `ValueError`, and the hardcoded `MODEL_DICT` allowlist) as GitHub issues or pull requests to the original `zjunlp/LightMem-Ego` repository, since these affect any user trying to use a non-OpenAI, non-Qwen3 model — this is the assistant's own recommendation, not something requested in the conversation.
-

23. Final Quick Start

text

```
# QUICK START

1. Activate environment
cd ~/Documents/fyp/LightMem-Ego/src/backend
source .venv/bin/activate

2. Start dependency/service (Ollama — usually already running via systemd)
systemctl status ollama # if inactive: sudo systemctl start ollama

3. Start embedding substitute servers
nohup python online_clip_embedding_server.py > clip_server.log 2>&1 &
disown
nohup python online_text_embedding_server.py > text_embed_server.log 2>&1 &
disown
sleep 10
curl http://127.0.0.1:18091/health
curl http://127.0.0.1:18096/health

4. Start API server
scripts/start_api.sh
sleep 5
curl http://127.0.0.1:8000/ping

5. Start all workers
scripts/start_online_all_workers.sh
sleep 20

6. Verify
timeout 8 python monitor_online_pipeline.py --watch
# Confirm every worker shows "ready"

7. Test
curl -s -X POST http://127.0.0.1:8000/stream/start \
  -H "Content-Type: application/json" \
  -d '{"input_mode": "frame_audio_stream"}'
# upload a frame, then POST /ask/{session_id}, then poll GET /query_task/{task_id}
```

text

```
# STOP
```

1. Stop AI harness workers

```
killall -9 -f "online_asr_worker.py"
killall -9 -f "online_visual_worker.py"
killall -9 -f "online_vlm2vec_embedding_server.py"
killall -9 -f "online_clip_embedding_server.py"
killall -9 -f "online_text_embedding_server.py"
killall -9 -f "online_memory_worker.py"
killall -9 -f "online_mst_consolidation_worker.py"
killall -9 -f "online_mst_refine_worker.py"
killall -9 -f "online_stream_worker.py"
killall -9 -f "online_live_ingest_worker.py"
killall -9 -f "online_query_worker.py"
killall -9 -f "online_worker.py"
```

2. Stop API server

```
killall -9 -f "api_server.py"
ps aux | grep -E "uvicorn" | grep -v grep # check for any leftover uvicorn processes, kill -9 <pid> if found
```

3. Stop other services (optional – Ollama can be left running)

```
sudo systemctl stop ollama
```